

OrbitTAMP: Grounding Language Models for Task and Motion Planning in Spacecraft Rendezvous

Yuji Takubo, Student Member, IEEE
Stanford University, Stanford, CA 94305, USA

Daniele Gammelli, Member, IEEE
Stanford University, Stanford, CA 94305, USA
Italian Institute of Artificial Intelligence (IIAI), Turin, Italy

Marco Pavone, Senior Member, IEEE
Stanford University, Stanford, CA 94305, USA
NVIDIA Research, Santa Clara, CA 95051, USA

Simone D'Amico, Member, IEEE
Stanford University, Stanford, CA 94305, USA

Abstract— Spacecraft rendezvous and proximity operations (RPO) are currently planned through an expertise-intensive process in which engineers translate high-level operational intent into safe, dynamically feasible trajectories, creating a bottleneck to scalable operations. Large language model (LLM)-based agents could offer an intuitive interface for this process, although their outputs are not inherently grounded in orbital dynamics, operational constraints, or the structure of admissible spacecraft maneuvers. To exploit their semantic reasoning while ensuring the generated plan's physical validity, this paper presents a hierarchical framework for spacecraft task-and-motion planning (TAMP) that grounds LLM reasoning in a graph of reusable behaviors and domain-specific planning modules. Within this framework, a pretrained LLM maps a natural-language command to a partial mission specification. The associated planners then resolve unspecified decisions within the admissible operational space. Finally, trajectory optimization converts the completed mission specification into a dynamically feasible trajectory. Numerical experiments demonstrate that this architecture substantially improves intent recovery over direct LLM generation, achieving 98% exact recovery of partial mission specifications across all evaluated splits when backed by frontier LLMs. Additional test-time-compute experiments show that, for a compact 9B model, verifier-guided revision increases exact recovery from 75% to 88%, while broader behavior-plan search independently improves selection among admissible trajectory realizations. Overall, these results establish a scalable and auditable foundation for language-driven agentic planning of spacecraft RPO.

Index Terms—Artificial Intelligence, Spacecraft Autonomy, Trajectory Optimization

Manuscript received XXXXX 00, 0000; revised XXXXX 00, 0000; accepted XXXXX 00, 0000.

Y. Takubo acknowledges the Ezoe Memorial Recruit Foundation for the financial support. (Corresponding author: Y. Takubo).

Y. Takubo, D. Gammelli, M. Pavone, and S. D'Amico are with the Department of Aeronautics and Astronautics. (e-mail: {ytakubo, gammelli, pavone, damicos}@stanford.edu).

Color versions of one or more of the figures in this article are available online at <http://ieeexplore.ieee.org>.

0018-9251 © 2020 IEEE

I. Introduction

The proliferation of distributed space systems, such as swarms and constellations, is increasing the need for scalable spacecraft operations. Rendezvous and proximity operations (RPO), in particular, require complex maneuver sequences for inspection, repair, refueling, and other servicing tasks. Traditionally, teams of operators and mission designers translate high-level mission intent or concepts of operations (CONOPS) into explicit trajectory specifications using their knowledge of astrodynamics, mission constraints, and available guidance modes. This dependence on specialized expertise limits the scalability of future spacecraft operations.

Large language models (LLMs) have the potential to establish natural language as an intuitive and broadly accessible interface between high-level mission intent and executable spacecraft trajectories. Specifically, recent advances in agentic LLM frameworks have demonstrated that foundation models (FMs) can interpret high-level instructions, decompose objectives into multistep plans, invoke external tools, and coordinate reasoning with action across interactive decision-making, autonomous scientific experimentation, and embodied operations [1], [2], [3]. For spacecraft RPO, however, semantic interpretation alone is insufficient. Language-model outputs must be grounded in spacecraft-specific behavior structures, operational constraints, and orbital dynamics before they can support safe execution. The central challenge is therefore not merely to translate language into a quantitative configuration, but to embed LLM reasoning within a planning architecture that converts incomplete intent into admissible and physically realizable spacecraft motion.

Consider the command: “*approach the target slowly from the $-V$ -bar, circumnavigate it for two orbits, and then depart to a 50 m along-track separation.*” To a human operator, this command conveys a clear maneuver intent through an ordered composition of familiar spacecraft behaviors: approach, circumnavigate, and depart. Such reusable behaviors provide abstractions for organizing complex motion, similar in function to *skills* [3] or *meta-actions* [4] in broader robotic systems. As a trajectory-planning specification, however, the command remains incomplete. It dictates several explicit requirements, including the two-orbit circumnavigation and final separation, while leaving other decisions such as phase durations, intermediate waypoints, and trajectory constraints unspecified. This distinction is central to spacecraft trajectory planning: conventional task and trajectory planners typically operate on formally defined actions, objectives, and constraints, whereas natural-language mission commands generally specify only a subset of this information, and the unstated remainder is where mission-design expertise must be applied. Grounding such a command therefore requires identifying the stated intent, resolving unspecified decisions within the admissible spacecraft operational space, and constructing a dynamically feasible and safe trajectory.

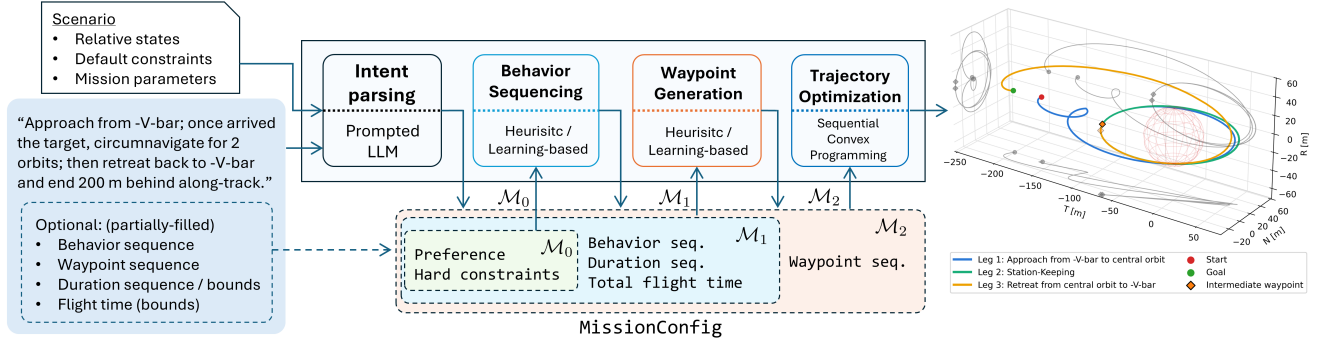


Fig. 1: Overview of the proposed framework. The intent parsing initializes the partial mission specification $\mathcal{M}_0$; behavior sequencing produces $\mathcal{M}_1$; waypoint generation produces $\mathcal{M}_2$; and trajectory optimization generates the final state and control trajectories. All three specifications are successive states of the same *MissionConfig*.

To address this problem, this paper introduces a hierarchical framework for grounding LLM reasoning for spacecraft task-and-motion planning (TAMP), illustrated in Fig. 1. Extending the preliminary concepts of structural decomposition for spacecraft behavior planning [5], [6], [7], the framework organizes the planning process into three stages: semantic grounding, completion of unspecified mission decisions, and physical trajectory verification. A pretrained LLM first maps the natural-language command to a partial mission specification that preserves the operator’s stated intent while explicitly representing unspecified information. A graph of reusable spacecraft behaviors, together with domain-specific planning modules, then completes the behavior sequence, timing, and waypoint decisions within the admissible operational space. Finally, trajectory optimization instantiates the resulting candidates as state-control trajectories and ensures their dynamical feasibility and safety. This hierarchical architecture grounds LLM reasoning at each stage while enabling multistage planning, computational tool use, and intermediate verification.

The central contribution of this paper is a hierarchical planning formulation in which operator underspecification is preserved and systematically completed, realized as an agentic architecture that scaffolds pretrained, off-the-shelf LLMs with structured, spacecraft-specific planning and verification modules. Because every stage completes the same partial specification, stated intent is preserved by construction rather than by prompt compliance and remains checkable throughout, while unstated decisions are resolved by the behavior graph and planning modules under orbital dynamics and safety constraints rather than left to the LLM. This design is motivated by both the scarcity of domain-specific training data for space applications [8] and the need for transparent, verifiable, and explicitly constrained decision-making in safety-critical spacecraft operations. The resulting architecture provides an auditable human–spacecraft interface and supports scalable language-driven planning for future distributed space systems. Specifically, the contributions of this work are as follows:

- A spacecraft RPO planning representation comprising a graph of reusable behaviors and a structured mission configuration that preserves partial operator specifications and supports hierarchical completion of behavior, timing, and waypoint decisions.
- An LLM-backed TAMP formulation that uses these representations to decouple intent parsing from trajectory realization, grounding semantic decisions in the admissible behavior structure and resolving unstated decisions within operational constraints and spacecraft dynamics.
- A stage-wise and end-to-end evaluation over 300 procedurally generated commands and multiple LLM backends and a deterministic parsing baseline that quantifies the effects of the grounding scaffold, model capability, and targeted test-time computation through verifier-guided revision and behavior-plan search.

The remainder of the paper is organized as follows. Sec. II compares the proposed framework with spacecraft planning, language grounding, and robotic TAMP. Sec. III defines the common mission representation, behavior graph, and systematic completion process. Sec. IV describes the on-orbit inspection case study and its specific implementations. Sec. V reports the module-level and end-to-end evaluation results. Finally, Sec. VI concludes the paper and discusses future work.

II. Related Work

This work builds on two closely related lines of research: spacecraft planning and trajectory generation, and language-grounded TAMP for robotic systems. The following review for each line of research is provided below.

A. Spacecraft Planning, Language Grounding, and Trajectory Optimization

Classical spacecraft autonomy separates high-level mission planning from guidance, navigation, and control.

Onboard systems have demonstrated activity planning, scheduling, execution monitoring, and replanning from predefined goals [9], [10]. Autonomous decision-making based on discrete logic has also been developed for formation flying and rendezvous and proximity operations [5], [11], [12]. In these systems, expert-defined activity models and transition logic specify high-level tasking and behavior sequencing, while waypoint and maneuver generation is generally handled by separate modules.

Trajectory optimization provides a principled mechanism for generating a state-control trajectory that enforces the underlying dynamics, waypoint conditions, and operational constraints [13], [14]. Learning-based methods can improve its numerical performance through warm-starting or approximation [15], [16], [17], [18]. These methods generally assume that the maneuver structure, waypoints, objectives, and constraints have already been specified based on the desired high-level spacecraft behaviors.

Recent work has explored the use of semantic reasoning for spacecraft operations [19], [8], [20]. Parallel research translates natural-language safety requirements into mathematical representations, including explicit constraints and signal temporal logic formulas [21], [22]. Prior work by the authors learned language-conditioned trajectory warm starts from paired command–trajectory data and refined them through sequential convex programming (SCP) [23].

Collectively, these approaches provide complementary capabilities for high-level spacecraft autonomy and continuous trajectory generation with potential for connection with semantics. Their interface nevertheless remains dependent on expert-specified maneuver definitions or complete trajectory-level representations, leaving no structured mechanism for preserving partial operator intent, resolving unspecified mission decisions, and verifying the resulting behavior sequence under orbital dynamics and safety constraints, in a scalable manner.

B. Learning-Guided Task and Motion Planning

TAMP integrates discrete task-level decision-making with continuous motion generation [24]. Classical TAMP evaluates symbolic choices through motion-level sampling or optimization [25], [26], whereas learning-guided approaches leverage data to prioritize task decisions, propose continuous parameters, or filter infeasible candidates [27], [28], [29]. These methods provide a general framework for coupling behavior selection with trajectory generation.

Language-conditioned planning in robotics provides a complementary perspective by enabling high-level instructions to guide planning and control. Existing approaches combine the reasoning capabilities of language models with learned affordances, executable programs, or external planners [3], [30], [31]. Some use LLM to propose or prioritize skill-level plans under feasibility models [32], while others translate instructions into a

structured formal task specifications used by downstream planners [33], [34].

Collectively, these methods demonstrate how language-level reasoning can be connected to executable motion through structured planning abstractions. However, their formulations generally seek a sufficiently specified symbolic problem or skill sequence and do not address the spacecraft-specific need to preserve operator underspecification while completing behavior, timing, and waypoint decisions within an admissible decision space for RPO missions.

Taken together, prior work in spacecraft mission planning, trajectory optimization, language grounding, and robotic TAMP provides complementary pieces towards language-driven spacecraft operations. The proposed framework unifies these directions by representing language as a partial mission specification and systematically completing its unspecified decisions through a graph of admissible spacecraft behaviors, trajectory-aware evaluation, and final trajectory optimization.

III. Methodology

The objective of this work is to generate an executable spacecraft trajectory from an operator’s high-level intent, expressed as a natural-language command. The planner must first infer an admissible sequence of spacecraft behaviors and then solve for the corresponding continuous trajectory. Because natural-language commands may omit relevant details or leave them underspecified, the framework must also identify and resolve missing information while preserving the operator’s stated intent.

The overall architecture is summarized in Fig. 1. The framework takes as input a natural-language command and a mission scenario specifying the initial relative state, default constraints, and other mission parameters. A shared representation, `MissionConfig`, carries the evolving mission specification throughout the pipeline. First, the intent parser maps the command to a partial specification, $\mathcal{M}_0$. The behavior-duration planning stage then uses the behavior graph to complete the behavior sequence and phase durations, yielding $\mathcal{M}_1$. Next, the waypoint planning stage completes the corresponding waypoint specifications to produce $\mathcal{M}_2$. Finally, trajectory optimization instantiates $\mathcal{M}_2$ as state-control trajectories subject to the adopted dynamics and safety constraints. Thus, $\mathcal{M}_0$, $\mathcal{M}_1$, and $\mathcal{M}_2$ represent successive stages in the completion of a single `MissionConfig`, with operator-specified information preserved throughout the pipeline.

The following subsection introduces the shared planning representations that support the proposed hierarchy. The planning modules are then described in sequence to show how these representations structure the admissible search space while preserving operator intent and ensuring that the resulting trajectory satisfies the adopted safety and dynamical feasibility requirements.

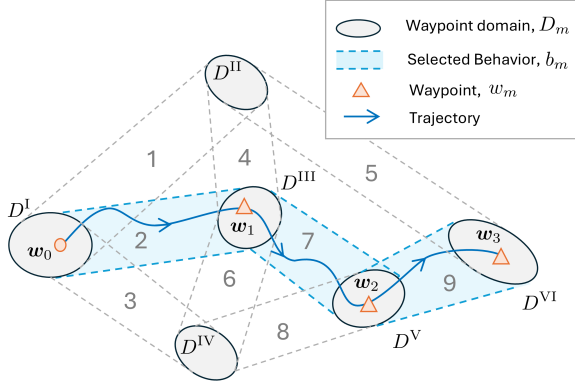


Fig. 2: Example behavior-waypoint graph.

A. Common Planning Representation

To make this planning problem tractable, this paper introduces two complementary abstractions: a graph-based representation of reusable spacecraft behaviors and a structured representation of the mission specification.

1. Behavior-Waypoint Graph

First, the trajectory is assumed to decompose into a sequence of *behavior primitives*, each representing an admissible transfer between prescribed waypoint domains over an allowable range of transfer times. Namely, this paper assumes that a trajectory can be represented as a sequence of waypoint-hopping transfers. This abstraction yields the behavior-waypoint graph $\mathcal{G}(\mathcal{D}, \mathcal{E})$, illustrated in Fig. 2, where $\mathcal{D}$ denotes the set of waypoint-domain nodes and $\mathcal{E} \subseteq \mathcal{D} \times \mathcal{B} \times \mathcal{D}$ denotes the set of behavior-labeled edges, with $\mathcal{B}$ representing the set of behavior labels. In the highlighted example, the behavior sequence $2 \rightarrow 7 \rightarrow 9$ induces the domain sequence $D^I \rightarrow D^{III} \rightarrow D^V \rightarrow D^{VI}$. Given the initial state w_0 , a particular instantiation selects waypoints $w_{1:3}$ from the corresponding domains, yielding a continuous state-control trajectory. Unlike previous works on kinodynamic motion-planning graphs [35], [36], the proposed behavior-waypoint graph operates at the CONOPS-level, representing continuous waypoint domains and admissible behavior transitions rather than discretized states or motion primitives.

2. Mission Configuration Interface

The second key abstraction is *MissionConfig*, a structured representation of the intermediate mission specification shared between the operator-facing interface and downstream planning modules. It comprises two fields encoding operator priorities and constraints and four planning fields that are progressively completed throughout the pipeline. These fields are defined below, where M denotes the number of behavior phases and $\perp$ denotes an unspecified entry:

- $\mathcal{P}$ (*preferences*): an operator-specified priority ordering over a predefined set of trajectory-quality metrics. The metrics are collected in $\mathbf{q} = [q_1, \dots, q_{N_{\mathcal{P}}}]^T$, and $\mathcal{P}$ is represented as a permutation of their

indices, with the first index denoting the highest-priority metric. The number of preference metrics is denoted as $N_{\mathcal{P}}$. This ordering is used to rank admissible candidates but does not alter their feasibility.

- $\mathcal{H}$ (*hard constraints*): operator-specified constraints that must be enforced by the downstream planner. Examples include bounds on time of flight, Δv budgets, or keep-out zone (KOZ) dimensions.
- $b_{1:M}$ (*behavior sequence*): an ordered sequence of behavior primitives selected from the behavior-waypoint graph, where each entry satisfies $b_m \in \mathcal{B} \cup \{\perp\}$ before completion.
- $d_{1:M}$ (*duration sequence*): the transfer-time specification associated with the behavior sequence. Each entry may be unspecified, fixed, or bounded, i.e., $d_m \in \{\perp\} \cup \mathbb{R}_+ \cup \{[d_m^-, d_m^+]\}$.
- T (*total flight time*): either specified by the operator, inferred from the duration sequence, or constrained by an admissible interval, i.e., $T \in \{\perp\} \cup \mathbb{R}_+ \cup \{[T^-, T^+]\}$.
- $w_{1:M}$ (*waypoint sequence*): the waypoint specification associated with each behavior primitive. A waypoint may be unspecified, specified as a symbolic domain, or specified as a possibly partial relative state, $w_m \in \{\perp\} \cup \mathcal{D} \cup (\mathbb{R} \cup \{\perp\})^6$.

Natural-language commands may express intent at different levels of granularity, ranging from explicit waypoint and timing specifications to qualitative behavioral and trajectory preferences. The alternative types supported by the duration, total flight time, and waypoint fields allow *MissionConfig* to preserve explicitly stated intent while marking unresolved decisions with $\perp$ for completion by downstream modules. This representation also aligns with established RPO operational practice, in which maneuver sequences are organized around predefined waypoints that serve as intermediate operational objectives [12].

B. Hierarchical Planning

With the two planning representations established, the following subsections describe each stage of the planning pipeline. The behavior-duration planning and waypoint generation modules expose common completion interfaces without prescribing a particular inference mechanism; therefore, each may be implemented using deterministic or learning-based methods. Regardless of the implementation, previously populated fields are preserved, and a completed mission specification is accepted only after its dynamical feasibility and safety have been verified through trajectory optimization.

1. Intent Parsing

The first module, the intent parser, maps the semantics of the input command to an initial *MissionConfig*. Using a curated prompt, a pretrained LLM translates the command into a structured JavaScript Object Notation (JSON) representation of the relevant mission semantics,

which is subsequently parsed into `MissionConfig`. The resulting initial mission specification, $\mathcal{M}_0$, assigns default values to unspecified preferences and hard constraints. Behavior, duration, and waypoint fields are populated only when directly supported by the command; all unresolved entries remain marked as $\perp$. An in-depth discussion of the LLM prompting procedure is provided in the Supplementary Material Sec. C.

2. Behavior-Duration Sequence Planning

The behavior-duration planner determines the sequence of spacecraft behaviors to be executed and the duration of each behavior. First, the planner enumerates all graph-valid behavior-and-duration plans consistent with the information specified in $\mathcal{M}_0$. Graph validity alone, however, does not guarantee that a candidate admits a feasible or desirable trajectory. Although each candidate could, in principle, be evaluated by completing its waypoint specification and solving the corresponding trajectory optimization problem, repeating this process for every candidate may be computationally prohibitive. The evaluator Q therefore approximates this downstream evaluation using domain-specific information:

$$[\hat{p}_{\text{conv}}, \hat{q}] = Q(s, \mathbf{b}_{1:M}, \mathbf{d}_{1:M}), \quad (1)$$

where $\hat{p}_{\text{conv}}$ estimates the probability of trajectory optimization convergence and serves as an empirical proxy for feasibility, while $\hat{q} \in \mathbb{R}^{N^{\mathcal{P}}}$ estimates the mission-performance metrics. Thus, Q enables candidate evaluation without requiring a separate full-horizon trajectory optimization solve for each plan. As discussed in Sec. IV, Q may be instantiated as an optimization-derived trajectory library, a learned surrogate trained on optimization-generated data, or another domain-specific evaluation model.

Formally, each candidate is an instantiated graph walk comprising behavior edges (D_{m-1}, b_m, d_m, D_m) , each paired with a transfer duration d_m , where $D_m \in \mathcal{D}$ and $b_m \in \mathcal{B}$. Starting from the known initial domain D_0 , the planner traverses admissible outgoing edges over the permitted number of phases and assigns each edge a discrete duration selected from its allowable range. Candidates inconsistent with behavior, waypoint domain, duration, total flight time, or other constraints already specified in $\mathcal{M}_0$ are discarded. This procedure yields the finite candidate set $\mathcal{C}(\mathcal{M}_0, s)$, whose elements have the form $c = (\mathbf{b}_{1:M}, \mathbf{d}_{1:M}, \mathbf{D}_{0:M})$. Candidates whose predicted convergence probability falls below the threshold $p_{\min}$ are then removed:

$$\mathcal{C}_{\text{pass}} = \{c \in \mathcal{C}(\mathcal{M}_0, s) \mid \hat{p}_{\text{conv}}(c) \geq p_{\min}\}. \quad (2)$$

The remaining candidates are ranked by the deterministic lexicographic map $\rho_{\mathcal{P}}(\hat{q})$ induced by the operator-specified preference ordering $\mathcal{P}$. Before comparison, the predicted metrics are ordered according to $\mathcal{P}$ and quantized at resolutions $\epsilon \in \mathbb{R}_{\geq 0}^{N^{\mathcal{P}}}$. Consequently, differences within the prescribed resolution for one metric defer the comparison to the next-highest-priority metric. The

selected candidate is therefore:

$$(\mathbf{b}_{1:M}^*, \mathbf{d}_{1:M}^*) = \arg \max_{(\mathbf{b}_{1:M}, \mathbf{d}_{1:M}, \mathbf{D}_{0:M}) \in \mathcal{C}_{\text{pass}}} \rho_{\mathcal{P}}(\hat{q}), \quad (3)$$

from which the behavior-duration $(\mathbf{b}_{1:M}^*, \mathbf{d}_{1:M}^*)$ sequence is used to complete $\mathcal{M}_1$ without modifying any entries inherited from $\mathcal{M}_0$.

3. Waypoint Generation

The third layer instantiates the waypoint sequence for the selected behavior-duration sequence $(\mathbf{b}_{1:M}^*, \mathbf{d}_{1:M}^*)$, thereby completing the mission specification and yielding $\mathcal{M}_2$. As with the evaluator Q , waypoint generation is problem-specific. The policy π_w defines a waypoint-placement strategy that may be implemented through direct optimization, a deterministic geometric or physics-informed heuristic, or a learned predictor:

$$\tilde{\mathbf{w}}_{1:M} = \pi_w(s, \mathbf{b}_{1:M}^*, \mathbf{d}_{1:M}^*). \quad (4)$$

The waypoint module proposes continuous relative-state waypoints $\tilde{\mathbf{w}}_{1:M}$ within the admissible domains $\mathbf{D}_{1:M}^*$ associated with the selected behavior primitives. These proposals are reconciled with any waypoint information supplied by the operator. If the operator specifies a symbolic waypoint domain, the proposed waypoint is accepted only if it lies within that domain. If the operator provides a partial relative state, the specified coordinates are preserved, and only the unspecified coordinates are completed. Waypoint generation therefore completes the partial specification without overwriting operator-provided information; the rule for selecting continuous states within the admissible domains depends on the particular implementation of π_w . In the subsequent trajectory optimization, the resulting waypoints may be imposed as fixed constraints or used as warm-start references.

4. Trajectory Optimization

Once `MissionConfig` is complete, its hard constraints, phase durations, and waypoint specifications are passed to the trajectory optimization module. This module computes nominal open-loop state and control trajectories subject to the adopted dynamics, waypoint, and safety constraints by solving the following discrete-time optimal control problem:

$$\min_{\{\mathbf{x}_k\}_{k=1}^N, \{\mathbf{u}_k\}_{k=1}^{N-1}} \mathcal{J} \quad (5a)$$

$$\text{subject to } \mathbf{x}_{k+1} = \mathbf{f}_k(\mathbf{x}_k, \mathbf{u}_k), \forall k \in \mathbb{Z}_{1:N-1}, \quad (5b)$$

$$(\mathbf{x}_k, \mathbf{u}_k) \in \mathcal{S}_k \cap \mathcal{H}_k, \forall k \in \mathbb{Z}_{1:N-1}, \quad (5c)$$

$$\mathbf{x}_1 = \mathbf{w}_0, \quad (5d)$$

$$\mathbf{x}_{k_m} \in \mathcal{W}_m, \quad \forall m \in \mathbb{Z}_{1:M}. \quad (5e)$$

Here, $\mathcal{J}$ denotes the objective function, N is the number of trajectory nodes, $\mathbf{x}_k$ and $\mathbf{u}_k$ denote the state and control input, respectively, $\mathbf{f}_k$ represents the discrete-time dynamics, and $\mathbb{Z}_{a:b} = [a, a+1, \dots, b]$. Equation (5b) enforces the system dynamics, while Eq. (5c) imposes the default safe set $\mathcal{S}_k$ and the augmented constraint set $\mathcal{H}_k$ defined from `MissionConfig`. Equation (5d) defines

the initial condition as w_0 . In Eq. (5e), k_m denotes the time step for the m -th waypoint, and $\mathcal{W}_m$ is its admissible waypoint domain generated from $\mathcal{M}_2$.

This paper considers two contracts for incorporating the generated waypoints into the optimal control problem. Under the *variable* contract, $\mathcal{W}_m = D_m^*$, allowing the waypoint state to vary within its selected admissible domain to improve trajectory feasibility and optimality. Under the *fixed* contract, $\mathcal{W}_m = \tilde{w}_m$, requiring the trajectory to pass through the waypoint proposed by the waypoint-generation module.

IV. Case Study: On-Orbit Inspection in Low Earth Orbit

The proposed architecture is demonstrated in an on-orbit inspection scenario in low Earth orbit, where a servicer spacecraft approaches a target spacecraft and performs close-proximity operations. Consistent with the hierarchical validation strategy, the case study presents the modules from downstream to upstream: trajectory optimization, waypoint generation and behavior-duration planning, and, finally, intent parsing. This ordering allows each lower-level module to be established before evaluating its upstream dependencies and, ultimately, the end-to-end pipeline.

The case study considers behavior sequences comprising two or three phases, such that $M \in 2, 3$. The physical scenario is parameterized by $s = [w_0, \alpha_0, r_{\text{KOZ}}]$, where w_0 denotes the initial state of the servicer relative to the target, $\alpha_0 = [a, e, i, \Omega, \omega, M_0]$ contains the target spacecraft's initial Keplerian orbital elements, and r_{KOZ} denotes the radius of the spherical keep-out zone (KOZ) enforced during trajectory optimization. The relative state is expressed using quasi-nonsingular relative orbital elements (ROE) [37], scaled by the target semimajor axis: $x = a[\delta a, \delta \lambda, \delta e_x, \delta e_y, \delta i_x, \delta i_y]$.

Scenario variation is introduced through the KOZ radius and the target's initial orbital phase. Specifically, r_{KOZ} is selected from 30, 40, 50 m, and the initial mean anomaly is selected from $M_0 \in \{0^\circ, 72^\circ, 144^\circ, 216^\circ, 288^\circ\}$. The remaining target orbital elements are fixed at $a = 6,738.1$ km, $e = 5.581 \times 10^{-4}$, $i = 51.6^\circ$, $\Omega = 301.0^\circ$, and $\omega = 26.2^\circ$. The trajectory is discretized using a time interval of $\Delta t = 900$ s with a maximum horizon of $N_{\text{max}} = 130$ time steps.

The following subsections define the preference metrics, behavior-waypoint graph, behavior-duration planning and waypoint-generation modules, and intent-parsing procedure used in the case study.

A. Preference Metrics

In this case study, $\mathcal{P}$ ranks $N_{\mathcal{P}} = 4$ metrics: fuel, flight time, effective observation exposure, and safety margin. Their definitions and the directions of optimality (i.e., minimizing or maximizing) are:

- **Fuel** (minimizing):

$$q_{\text{fuel}} = \sum_{k=1}^{N-1} \|u_k\|_2, \quad (6)$$

where u_k denotes the impulsive control input.

- **Flight time** (minimizing):

$$q_{\text{time}} = \sum_{m=1}^M d_m. \quad (7)$$

- **Effective observation exposure** (maximizing): The accumulated proximity-based opportunity proxy is computed as

$$q_{\text{obs}} = \frac{\Delta t}{2} \sum_{k=1}^{N-1} [g(\rho_k) + g(\rho_{k+1})], \quad (8)$$

where ρ_k is the inter-satellite range at node k , and

$$g(\rho) = \max \left\{ 0, 1 - \frac{|\rho - (r_{\text{KOZ}} + \bar{\rho})|}{\bar{\rho}} \right\}, \quad (9)$$

where $\bar{\rho} = 25$ m in this case study.

- **Safety margin** (maximizing):

$$q_{\text{safe}} = \min_k \rho_k - r_{\text{KOZ}}. \quad (10)$$

B. Behavior-Waypoint Graph

Table I enumerates the waypoint domain defined in the ROE space. For all waypoints, $\delta a = 0$ is imposed to eliminate secular drift in the along-track direction. In addition, $\delta e_x = \delta i_x = 0$ is imposed to mitigate secular drift from the J_2 -perturbation while maintaining radial-normal (RN)-plane separation through eccentricity/inclination (E/I)-vector separation [37]. The waypoint space is therefore reduced to $a[\delta \lambda, \delta e_y, \delta i_y] \in \mathbb{R}^3$. For the E/I-separated domains (E_0 , E_+ , and E_-), wide RN-plane separation is enforced. Note that the intervals of $a\delta e_y$ and $a\delta i_y$ instantiate four disconnected subdomains admissible for E/I-vector separation [37]. In contrast, flat domains (F_+ and F_-) are located around the along-track axis (i.e., $\pm V$ -bar) with almost no RN-plane separation. The behavior primitives in Table II define transfers between waypoint domains; given (D_{m-1}, b_m, d_m) , the destination domain D_m is unique. The corresponding admissible duration ranges are also listed in the table. The adopted graph is not fully connected because of the operational constraints. Despite its compact symbolic representation, the graph contains five waypoint domains and 13 behavior-primitive types, which instantiate 21 directed edges and induce 466 graph-valid two- or three-phase paths. For dataset generation, the three variable components of the initial relative state, $[a\delta \lambda, a\delta e_y, a\delta i_y]$, are sampled within the corresponding waypoint domain (cf. Table I) using a discrete grid for training and continuous uniform sampling for held-out evaluation. For each behavior, five duration values are uniformly spaced over the admissible range in Table II. Combining these paths with the discretized duration choices available in each

TABLE I: Waypoint domains used in the case study.

Domain	Description	$a\delta\lambda$ [m]	$a\delta e_y$ [m]	$a\delta i_y$ [m]
E_0	E/I-separated, Central	$[-5, 5]$	$\pm[30, 70]$	$\pm[30, 70]$
E_+	E/I-separated, +V-bar	$[100, 250]$	$\pm[30, 70]$	$\pm[30, 70]$
E_-	E/I-separated, -V-bar	$[-250, -100]$	$\pm[30, 70]$	$\pm[30, 70]$
F_+	Flat, +V-bar	$[100, 250]$	$[-5, 5]$	$[-5, 5]$
F_-	Flat, -V-bar	$[-250, -100]$	$[-5, 5]$	$[-5, 5]$

TABLE II: Behavior primitives used as directed edges in the behavior-waypoint graph.

ID	Behavior primitive	Directed transitions	Duration (orbits)
1	Station-keeping	Self-loop at all domains	$[1, 3]$
2	Drift, slow short, +V direction	$E_0 \rightarrow E_+, E_- \rightarrow E_0$	$[2, 4]$
3	Drift, slow short, -V direction	$E_0 \rightarrow E_-, E_+ \rightarrow E_0$	$[2, 4]$
4	Expand R/N separation	$F_+ \rightarrow E_+, F_- \rightarrow E_-$	$[1, 3]$
5	Shrink R/N separation	$E_+ \rightarrow F_+, E_- \rightarrow F_-$	$[1, 3]$
6	Approach from -V-bar	$F_- \rightarrow E_0$	$[1.5, 4]$
7	Retreat to +V-bar	$E_0 \rightarrow F_+$	$[1.5, 4]$
8	Approach from +V-bar	$F_+ \rightarrow E_0$	$[1.5, 4]$
9	Retreat to -V-bar	$E_0 \rightarrow F_-$	$[1.5, 4]$
10	Drift, fast long, +V direction	$F_- \rightarrow F_+$	$[1.5, 3.5]$
11	Drift, fast long, -V direction	$F_+ \rightarrow F_-$	$[1.5, 3.5]$
12	Drift, slow long, +V direction	$E_- \rightarrow E_+$	$[4, 6]$
13	Drift, slow long, -V direction	$E_+ \rightarrow E_-$	$[4, 6]$

phase gives $89 \times 5^2 + 377 \times 5^3 = 49,350$ discrete behavior-duration plans before accounting for discrete scenario variation and continuous waypoint domains.

C. Trajectory Optimization

As a specific instantiation of Eq. (5), the following trajectory optimization problem is formulated:

$$\min_{\{\mathbf{x}_k\}_{k=1}^N, \{\mathbf{u}_k\}_{k=1}^{N-1}} \sum_{k=1}^{N-1} \|\mathbf{u}_k\|_2 \quad (11a)$$

$$\text{subject to } \mathbf{x}_{k+1} = \Phi_k(\mathbf{x}_k + \Gamma_k \mathbf{u}_k), \forall k \in \mathbb{Z}_{1:N-1}, \quad (11b)$$

$$\mathbf{x}_{k0}^\top S_{kl} \mathbf{x}_{k0} \geq 1, \quad \begin{matrix} \forall k \in \mathbb{Z}_{1:N-1}, \\ \forall l \in \mathbb{Z}_{1:N_s} \end{matrix} \quad (11c)$$

$$\mathbf{x}_1 = \mathbf{w}_0, \quad (11d)$$

$$\mathbf{x}_{km} \in \mathcal{W}_m, \quad \forall m \in \mathbb{Z}_{1:M} \quad (11e)$$

$$\text{where } \Phi_k = \Phi(t_{k+1}, t_k), \quad (11f)$$

$$\Phi_{kl} = \Phi(t_k + \tau_l, t_k), \quad (11g)$$

$$\Psi_{kl} = \Psi(t_k + \tau_l), \quad (11h)$$

$$S_{kl} = \Phi_{kl}^\top \Psi_{kl}^\top P \Psi_{kl} \Phi_{kl}, \quad (11i)$$

$$\mathbf{x}_{k0} = \mathbf{x}_k + \Gamma_k \mathbf{u}_k. \quad (11j)$$

Discrete-time dynamics are expressed in Eq. (11b), where $\Phi(t_{k+1}, t_k) \in \mathbb{R}^{6 \times 6}$ is the state transition matrix of ROE with the secular J_2 effect [38], and $\Gamma_k = \Gamma(t_k) \in \mathbb{R}^{6 \times 3}$ is the control input matrix [37], and $\mathbf{u}_k$ is an impulsive velocity change in the target's radial-tangential-normal (RTN) frame. The nonconvex passive safety constraint in Eq. (11c) is imposed at the prescribed controlled-trajectory nodes t_k and post-failure drift evaluation times $\{\tau_l\}_{l=1}^{N_s} \subset [0, \tau^s]$ after an abrupt loss of control. $\Psi(t) \in \mathbb{R}^{6 \times 6}$ is the first-order mapping from ROE to the relative position and velocity resolved in the RTN frame [37].

The matrix $P = \text{diag}[\mathbf{I}_3/r_{\text{KOZ}}^2, \mathbf{0}_3] \in \mathbb{R}^{6 \times 6}$ defines the geometry of the spherical KOZ. The passive safety horizon is set to $\tau^s = 1$ orbit and sampled at $N_s = 50$ equally spaced timesteps. Additional considerations such as chance constraints [39], [18], and continuous-time safety constraints [40], [23] can be incorporated without changes in the upstream components; these extensions are outside the scope of this case study. Overall, this class of minimum-fuel problems with safety and waypoint constraints is generally applicable to a wide range of rendezvous guidance and control.

This paper adopts the SCP algorithm SCvx* [41] to solve the problem during both dataset generation and deployment. The hyperparameters of the algorithm are summarized in the Supplementary Material Sec. F.

D. Behavior-Duration Planning and Waypoint Generation

The case study considers deterministic and learning-based realizations of the two planning modules introduced in Sec. III. For behavior-duration planning, candidate quality is obtained either from an optimization-derived leg library or from a learned metric surrogate Q_ψ . For waypoint generation, continuous states are selected using either a representative-waypoint heuristic or a learned policy p_ϕ . The following subsections describe these realizations.

1. Coupled Behavior-Waypoint Realization

A simple heuristic selects representative waypoints at the midpoint of each admissible domain in Table I, despite its suboptimality. This reduces the problem to joining pre-computed legs between representative waypoints, making candidate screening and ranking fast. A representative waypoint is placed at the midpoint of each connected component of the E/I-separated domains (i.e., E_0 , E_+ , and E_-), and at the midpoint of each flat domain (i.e., F_+ and F_-).

An optimization-derived leg library is constructed offline by solving Eq. (11) for every admissible transfer between representative waypoints, duration grid, target semimajor axis, and KOZ radius considered in the case study. Each library entry records whether the transfer converges and its associated trajectory-quality metrics. In particular, the library assumes a circular Keplerian orbit ($e = 0$ and no J_2 perturbations), reducing the target's orbit parameterization to its semimajor axis instead of the full six-dimensional state. Note that this assumption could induce deployment error. However, this approximation keeps the library compact, with only 2,160 entries across the three KOZ radii at the nominal semimajor axis.

At deployment, each behavior-duration candidate in $\mathcal{C}(\mathcal{M}_0, s)$ is expanded over its admissible representative-waypoint chains. Because the measured initial state generally does not coincide with a representative waypoint, the first leg of each chain, from the current state to the first

representative waypoint, is solved online using SCP. All subsequent legs are retrieved from the offline leg library.

A chain is retained only if the online first-leg solve and all required library entries are feasible. The corresponding leg-level metrics are then composed to rank them: fuel consumption, flight time, and effective observation exposure are summed across the legs, whereas the safety margin is given by the minimum leg-level value.

This composition provides an auditable mechanism for candidate screening and ranking. It is distinct from the final full-horizon trajectory optimization, which receives the selected waypoint sequence and generates the final verified trajectory.

2. Heuristic Waypoint Generation

The heuristic policy constructs the waypoint sequence incrementally based on the same representative waypoint set used in the coupled behavior–waypoint realization. For an E/I-separated destination domain, the heuristic selects the connected component whose midpoint is closest to the preceding reference state, whereas the along-track coordinate is placed at the midpoint of the destination interval; this preserves the sign combination of $(\delta e_y, \delta i_y)$ at the previous waypoint. Flat-domain waypoints are placed at their domain centers.

3. Learning-Based Waypoint Generation

The learning-based waypoint policy is the conditional model $p_\phi(\mathbf{w}_{1:M} \mid s, \mathbf{b}_{1:M}, \mathbf{d}_{1:M})$, where ϕ denotes the model parameters. The model is supervised using the optimization-derived converged trajectories. The output parameterization of the model is carefully designed so that it guarantees that decoded waypoints remain within the graph-prescribed waypoint domains, even for domains that support disconnected subdomains, such as E/I-separated domains (i.e., E_0 , E_+ , and E_-). Details on network architecture, parameterization, and loss function are provided in the Supplementary Material Sec. A.

4. Learning-Based Behavior-Duration Planning

Learning-based behavior-duration planning uses a surrogate model Q_ψ , with parameters ψ , to predict the convergence of trajectory optimization and metric outcomes of a candidate behavior-duration sequence. The training dataset is generated by randomizing the scenario and behavior-duration sequence, applying a specified waypoint-generation policy π_w , and solving the trajectory optimization. Although flight time is a deterministic function of the behavior-duration sequence in this case study, the surrogate predicts all four preference metrics for the uniform evaluation of the metric-prediction interface. At deployment, graph-valid candidates are evaluated in a batched forward pass, candidates below the convergence threshold are removed, and the remainder are ranked using the same deterministic preference map $\rho_{\mathcal{P}}$ to determine the top-ranked behavior-duration plan. Details on the architecture, parameterization, and loss function are provided in the Supplementary Material Sec. B.

E. Intent Parsing

The intent parser uses a pretrained off-the-shelf LLM to map an operator command into the partial mission specification $\mathcal{M}_0$ as a structured output. The prompt provides the behavior–waypoint ontology and output schema, and instructs the model to populate only information explicitly supported by the command, leaving unspecified fields at $\perp$. Because populated entries are preserved by downstream planning stages, this conservative parsing rule avoids prematurely completing the mission. Details on the prompt development are given in the Supplementary Material Sec. C.

A central challenge in evaluating intent parsing is establishing an unambiguous reference for the operator’s intended $\mathcal{M}_0$. Evaluation commands are therefore generated *label-first*: a mission intent and corresponding reference specification $\mathcal{M}_0^*$ are constructed, and then the command is rendered systematically using family-specific templates. No language model participates in either label construction or command generation, providing an exactly known reference and avoiding circular evaluation. The sampled mission families span maneuver composition, quantitative constraints, preferences, and expert overrides.

Based on the label-first command generation, three 100-command splits are developed: *development*, *held-out instance*, and *held-out structural* to develop the structured prompts for the LLM and to evaluate the intent parsing capability in the following section. The development split is used for prompt development and contains commands rendered using the nominal family-specific linguistic templates. The held-out-instance split contains independently sampled mission specifications and behavior compositions expressed using the same linguistic styles as the development split, thereby evaluating generalization to unseen mission instances without introducing an additional language-distribution shift. The held-out-structural split similarly remains within the supported mission families and behavior–waypoint ontology, but additionally renders the commands using an unseen terse radio-communication register. This split therefore evaluates sensitivity to changes in linguistic structure while preserving the underlying planning vocabulary and mission semantics. The details of the commands’ linguistic styles are summarized in the Supplementary Material Sec. D.

At deployment, the parser may abstain when no valid specification is obtained. A graph-validity guard rejects behavior–waypoint combinations inconsistent with the graph before downstream planning.

V. Results

This section evaluates the proposed framework along three axes: architectural scaffolding, model capability, and test-time computation. Each axis corresponds to a design decision that arises when deploying a natural-language

interface for spacecraft operations and therefore warrants evaluation beyond the architecture itself.

The first axis concerns how domain knowledge is incorporated into the system. Because orbital dynamics, operational constraints, and mission-design conventions may not be reliably encoded within a general-purpose LLM, the proposed framework externalizes this knowledge through a structured mission representation, a behavior–waypoint ontology, and dedicated planning modules. Thus, the distinct contributions of these components and of the resulting hierarchical decomposition must be evaluated separately.

The second axis concerns the capability of the underlying language model. Candidate backends range from compact models suitable for onboard or resource-constrained deployment to frontier models intended primarily for ground-based planning. Accordingly, the evaluation must consider not only overall performance but also how the nature of failure changes with model capability, including the violation of the structured interface and misinterpretation of the intent.

The third axis concerns the allocation of computation at inference time. Additional test-time computation devoted to reasoning, verification, or search has been shown to substantially improve performance without requiring model retraining [42]. Within the proposed hierarchy, this computation may be assigned to semantic refinement during intent parsing or candidate search during behavior planning. As they address different failure modes, whether their gains are complementary determines how a fixed computational budget should be distributed across the pipeline.

Together, these axes motivate the following research questions, which are evaluated at the level of both the individual planning modules and the complete language-to-trajectory pipeline:¹

- 1) **RQ1 (Architecture):** What are the distinct contributions of architectural scaffolding and hierarchical planning to language-to-trajectory generation?
- 2) **RQ2 (LLM Capability):** How does LLM capability affect structured intent recognition, failure modes, and end-to-end mission satisfaction?
- 3) **RQ3 (Test-time Compute):** How does the additional test-time computation across the planning hierarchy affect intent reliability and realized trajectory preference?

A. Module-Level Validation and Representative End-to-End Realization

Before addressing the three RQs, this subsection validates candidate realizations of the downstream planning layers and illustrates their integration within the complete architecture. Specifically, alternative waypoint-generation

and behavior–duration-planning modules are compared, followed by a representative end-to-end realization showing how the initially partial `MissionConfig` is progressively completed across the planning hierarchy from a natural-language command.

1. Waypoint Generation

First, waypoint generation is evaluated on 1,000 cases sampled from the held-out test dataset. No language model or behavior-duration selection is involved in this experiment; only the waypoint proposal and the waypoint constraint handling in the subsequent SCP are varied. This is the held-out 10% split of the on-grid dataset in Sec. IV-D3, not a continuous-domain test. The behavior–duration sequence and scenario parameters are fixed for each case. Three waypoint proposals are compared: a random point in each destination domain, the minimum-displacement heuristic from π_w^{greedy} , and the prediction of p_ϕ . For the random proposal, a station-keeping phase uses the simple behavior-consistent rule of carrying the preceding waypoint forward rather than sampling a new waypoint. As discussed in Eq. (5e), each waypoint proposal is evaluated under two waypoint contracts in the trajectory optimization: *fixed* and *variable*. Table III summarizes downstream SCP success, fuel cost, and iteration count. Under the variable-waypoint contract, both structured waypoint proposals achieve approximately 95% SCP success and substantially lower fuel cost than random initialization, indicating that the waypoint-generation module provides useful continuous realizations of the selected behavior sequence. Under the fixed contract, the minimum-displacement heuristic attains the highest success rate, whereas the learned proposal yields lower fuel among converged cases but a lower success rate. Based on these results, the minimum-displacement heuristic with variable-waypoint SCP is adopted in the subsequent experiments as a simple, effective, and auditable realization of the waypoint generation interface.

2. Behavior-Duration Planning

Behavior-duration planning is evaluated by whether surrogate-guided ranking identifies useful candidates from the graph-valid set without exhaustive downstream trajectory optimization. The leg-library and learned surrogate Q_ψ realizations use different waypoint contracts and are therefore evaluated independently rather than compared directly. For each realization, the structured selector is compared against an offline best-of-64 random-search reference: 64 uniformly sampled graph-valid behavior-duration candidates are evaluated using full-horizon SCP under the same waypoint contract, and the best converged trajectory is retained according to the realized preference. Although computationally impractical for deployment, this procedure provides a strong reference for random selection under a fixed evaluation budget.

The evaluation considers 100 scenarios. Under the fixed-waypoint contract, leg-library selection is evaluated against random search using the representative waypoints.

¹All experiments were conducted on a single NVIDIA GeForce RTX 4090 GPU with an AMD Ryzen 9 7950X CPU.

TABLE III: Waypoint-generation performance on 1,000 held-out test cases; fuel and SCP iterations are the mean $\pm$ standard deviation over the cases in which the SCP converges.

	Random		Heuristic		Learned p_ϕ	
	Fixed	Variable	Fixed	Variable	Fixed	Variable
SCP success [%]	84.0	95.0	95.7	95.1	77.1	95.6
Fuel [cm/s]	15.31 $\pm$ 8.83	11.05 $\pm$ 5.37	9.37 $\pm$ 5.42	6.73 $\pm$ 4.19	7.00 $\pm$ 4.08	6.65 $\pm$ 4.13
SCP iterations	6.86 $\pm$ 8.01	9.98 $\pm$ 7.43	5.85 $\pm$ 7.25	8.05 $\pm$ 6.13	6.72 $\pm$ 7.27	7.21 $\pm$ 7.12

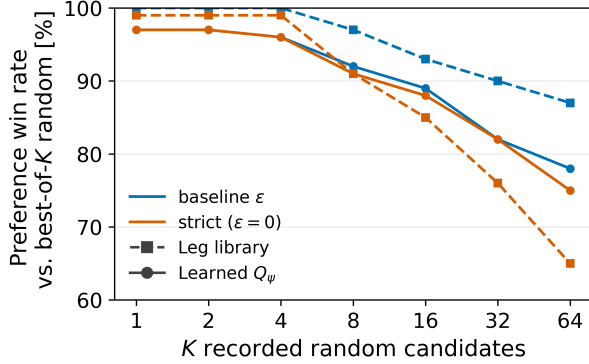


Fig. 3: Preference win rate of the structured behavior–duration planning against the best of K uniformly sampled graph-valid candidates over 100 scenarios.

By contrast, under the variable-waypoint contract, Q_ψ selection is evaluated against random search using the minimum-displacement heuristic to initialize variable-waypoint SCP. Note that the evaluated Q_ψ is the surrogate trained using the minimum-displacement waypoint policy and is therefore assessed under the corresponding variable-waypoint trajectory-optimization configuration. The scenario parameters are sampled from the case-study domain defined in Sec. IV. The lexicographic preference is balanced across four priority orders, with fuel, flight time, observation, and safety margin each serving as the highest-priority metric in 25 scenarios while the remaining metrics retain their default order.

Figure 3 reports how often each structured selector outperforms the best-of- K random candidates for $K \leq 64$. For each scenario, the K candidates are sampled from the recorded pool of 64 and ranked under the realized preference. Sensitivity to the lexicographic margins is evaluated using both strict lexicographic ranking, $\epsilon \simeq 0$, and the baseline margin $\epsilon = [1 \text{ cm/s}, 900 \text{ s}, 1 \text{ s}, 5.0 \text{ m}]$. Table IV summarizes the resulting trajectory performance. Both structured realizations maintain near-universal downstream SCP convergence and select candidates near the top of their corresponding random-search distributions. The leg-library realization achieves mean candidate percentiles of 98.9% and 99.6% under the strict and baseline margins, respectively, while Q_ψ achieves 96.2% and 96.5%. The best-of-64 random baseline is an offline quality reference requiring 64 full-horizon SCP evaluations per scenario, whereas the auxiliary-call column counts only selector-side first-leg

SCP solves, of which the leg-library realization requires 60 per scenario (6,000 across 100 scenarios). Overall, both domain-specific realizations of Q effectively prioritize high-quality behavior-duration candidates without exhaustively solving the full candidate set.

3. Representative End-to-End Realization

Table V traces a representative command through the shared mission representation. The command specifies the ordered maneuver structure, the one-orbit near-target phase, the total-time bound, and the priority of fuel over observation, while leaving the remaining phase durations and continuous waypoints unspecified. These requirements are preserved in $\mathcal{M}_0$. The behavior–duration planner completes the sequence with $\mathbf{d} = [2.5, 1.0, 1.5]$ orbits, satisfying the five-orbit bound, and the waypoint module produces the corresponding initialization in $\mathcal{M}_2$. Full-horizon SCP subsequently adjusts these waypoints while preserving their behavioral interpretation and enforcing the adopted dynamics and safety constraints. The resulting trajectory is shown in Fig. 4.

B. RQ1: Effects of Structured Scaffolding

This subsection validates the structured scaffolding by performing the architectural ablation of the scaffolded LLM-based TAMP pipeline. Based on the module-level results in the previous subsection, the subsequent end-to-end experiments adopt the learned surrogate Q_ψ for behavior-duration planning, the minimum-displacement heuristic for waypoint generation, and the variable-waypoint contract for final trajectory optimization, unless otherwise stated. The complete language-conditioned pipeline is evaluated to determine whether high-level operator intent can be translated into a dynamically feasible trajectory while remaining consistent with the reference mission specification. Pipeline completion requires the planning process to terminate with a converged SCP trajectory, whereas reference- $\mathcal{M}_0^*$ satisfaction additionally requires the resulting plan and trajectory to preserve the operator-specified information contained in the reference partial mission specification.

To isolate the effect of planning structure, the proposed hierarchical architecture is compared against two direct-generation baselines using the same GPT-5.6 Terra (high reasoning effort) [43]. The proposed architecture follows the full hierarchy described above: the LLM first generates the partial specification $\mathcal{M}_0$, after which Q_ψ -based behavior-duration planning and minimum-

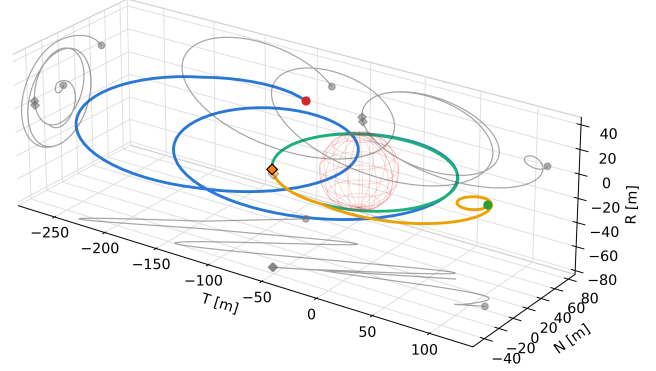
TABLE IV: Behavior-duration planning and corresponding trajectory performance after trajectory optimization. Fuel and SCP iterations are the mean $\pm$ standard deviation over successful trajectories.

Behavior selector	Waypoint proposal	Waypoint contract in SCP	SCP success	Fuel [cm/s]	SCP iterations	Aux. SCP calls/request	Percentile [%]		Online time [s]
							Strict	Baseline	
Uniform random (best of 64)	Representative waypoint	Fixed	100/100	7.65 ± 7.04	2.90 ± 3.80	0	—	—	—
Leg library			100/100	5.12 ± 8.06	3.37 ± 7.37	60	98.9	99.6	74.77
Uniform random (best of 64)	Min.-displacement heuristic	Variable	100/100	3.82 ± 3.94	4.75 ± 4.30	0	—	—	—
Learned Q_ψ			97/100	2.07 ± 2.79	3.57 ± 3.93	0	96.2	96.5	3.02

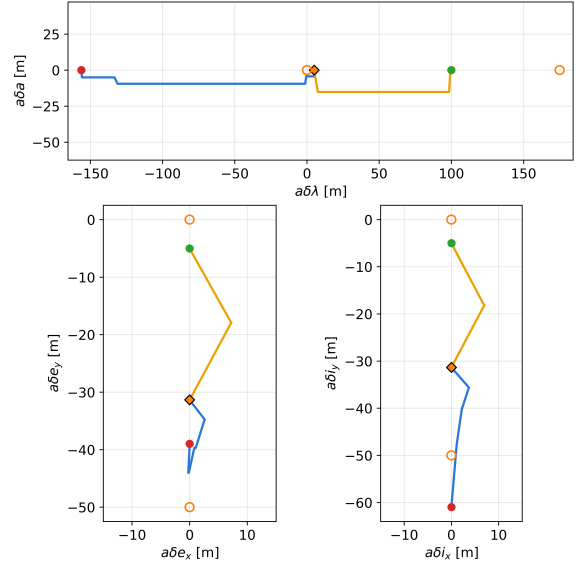
TABLE V: Representative progressive completion of a natural-language command into $\mathcal{M}_0$, $\mathcal{M}_1$, and $\mathcal{M}_2$, followed by the SCP-optimized waypoint sequence.

Entry	Representative realization
Command	Drift towards the target, traverse the near-target region for 1 orbit, and then withdraw toward +V-bar. Prioritize minimizing fuel over maximizing observation quality, and complete the entire operation within 5 orbits.
Scenario	$w_0 = [0, -156.24, 0, -38.97, 0, -60.99]$ $r_{\text{KOZ}} = 30$ m (default), $M_0 = 0.0^\circ$
$\mathcal{M}_0$	$\mathcal{P} = [\text{fuel}, \text{observation}]$, $\mathcal{H} = \emptyset$ (default) $T \in [\perp, 5.0]$ orbits $\mathbf{b} = [2, 1, 7]$, $\mathbf{d} = [\perp, 1.0, \perp]$ orbits, $\mathbf{w} = [\perp, E_0, F_+]$
$\mathcal{M}_1$	$\mathbf{b} = [2, 1, 7]$ (Slow, short drift in the +V direction → station keeping → retreat to +V-bar) $\mathbf{d} = [2.5, 1.0, 1.5]$ orbits, $T = 5.0$ orbits
$\mathcal{M}_2$	Warm-start waypoints [m]: $\tilde{\mathbf{w}}_1 = [0, 0, 0, -50, 0, -50]$, $\tilde{\mathbf{w}}_2 = [0, 0, 0, -50, 0, -50]$, $\tilde{\mathbf{w}}_3 = [0, 175, 0, 0, 0, 0]$.
$\mathbf{w}^*$	SCP-optimized waypoints [m]: $\mathbf{w}_1^* = [0, 5.00, 0, -31.33, 0, -31.35]$, $\mathbf{w}_2^* = [0, 5.00, 0, -31.33, 0, -31.35]$, $\mathbf{w}_3^* = [0, 100.00, 0, -5.00, 0, -5.00]$.

displacement waypoint generation complete $\mathcal{M}_1$ and $\mathcal{M}_2$. The *Direct- $\mathcal{M}_2$* baseline is provided with the same `MissionConfig` schema, behavior-waypoint graph, and physical scenario, but directly generates the completed $\mathcal{M}_2$ in a single LLM stage, thereby bypassing the intermediate behavior-duration search and trajectory-aware candidate evaluation. The *unscaffolded* baseline receives the operator command and physical scenario, but is not provided with the `MissionConfig` schema, behavior, waypoint domains, or behavior-waypoint graph; it instead directly proposes numerical durations and ROE waypoints. For all three architectures, the resulting planning specification is subsequently evaluated through the same full-horizon trajectory optimization module. The hierarchical architecture is limited to 900 completion tokens per LLM call, whereas the two direct-generation baselines are allowed up to 3,000 completion tokens.



(a) RTN frame



(b) ROE space. Red circles indicate the proposed waypoints used for the warm-starting in the SCP.

Fig. 4: Representative trajectory optimized via SCP based on the completed `MissionConfig` (cf. Table V).

Each baseline is compared pairwise against the proposed hierarchical architecture according to two outcomes: (i) whether each architecture produces a trajectory aligned with the reference mission intent and (ii),

when both architectures are intent-aligned, which realized trajectory is preferred under the lexicographic preference mapping. Accordingly, Fig. 5 reports the pairwise outcome of each baseline against the proposed hierarchical architecture across the three dataset splits. The bar chart reports whether both architectures are intent-aligned, only one is intent-aligned, or neither is intent-aligned. For cases in which both are intent-aligned, the figure further distinguishes whether the hierarchical trajectory is preferred, the comparator trajectory is preferred, or the two are equivalent under the realized preference mapping.

Across all three datasets, the frontier model rarely produces a realization consistent with the reference mission intent without the operational scaffold. Under its fixed 3,000-token generation limit, the unscaffolded planner exhausted the token limit for 45, 44, and 69 commands on the development, held-out-instance, and held-out-structural splits, respectively; these empty responses are counted as operational failures. In contrast, providing the `MissionConfig` and behavior-waypoint ontology substantially recovers intent alignment, indicating that the structured representation itself plays an important grounding role rather than serving only as an implementation interface. However, when both the proposed architecture and the Direct $\mathcal{M}_2$ baseline produce intent-aligned trajectories, the former realizes the operator's preferred trajectory more frequently in all three dataset splits. This result suggests that direct structured generation is often sufficient to identify an admissible mission realization, whereas explicit candidate enumeration and trajectory-aware evaluation provide a mechanism for selecting among multiple admissible alternatives according to feasibility and operator preferences. Conversely, the few intent-aligned trajectories produced by the unscaffolded baseline can be competitive in trajectory quality, indicating that its primary deficiency is operational grounding rather than numerical trajectory generation.

C. RQ2: Effect of LLM Capability

This subsection examines how the capability of the off-the-shelf LLM backend affects intent parsing and end-to-end trajectory generation. Three backends spanning two deployment regimes are evaluated: the compact open-weight models Qwen3.5-9B [44] and Ministral3-8B-Instruct [45], and the representative frontier model GPT-5.6 Terra with high reasoning effort. The compact models are loaded using 8-bit quantization, resulting in an approximately 10 GB inference memory footprint in the adopted implementation and motivating their consideration for resource-constrained deployment.

Using the label-first benchmark introduced in Sec. IV-E, the intent-parsing prompt is developed using the 100-command development split and frozen before evaluation. Performance is reported on this split for reference and on two independent 100-command held-out splits. The held-out-instance split contains newly sampled mission specifications and behavior compositions expressed using

TABLE VI: Intent-parsing performance on the prompt-development split and two held-out splits.

Dataset split	Backend	Exact $\mathcal{M}_0$ [%]	Atomic match [%]	
			Precision	Recall
Development	Qwen3.5-9B	75	90.6	85.1
	Ministral3-8B-Instruct	70	88.1	89.0
	GPT-5.6 Terra (high)	98	99.0	99.5
Held-out instance	Qwen3.5-9B	78	91.1	89.8
	Ministral3-8B-Instruct	70	89.1	90.3
	GPT-5.6 Terra (high)	98	99.5	99.0
Held-out structural	Qwen3.5-9B	70	91.7	82.9
	Ministral3-8B-Instruct	71	90.1	94.5
	GPT-5.6 Terra (high)	98	98.0	98.0

the linguistic styles represented during prompt development. The held-out-structural split additionally renders the commands using an unseen terse radio-communication register. Both held-out splits remain within the supported mission families and behavior-waypoint ontology.

The intent-parsing module is evaluated using exact- $\mathcal{M}_0$ match together with atomic precision and recall. Atomic precision is the fraction of populated atomic fields in the parsed $\mathcal{M}_0$ (including preferences, hard constraints, behaviors, durations, and waypoints) that agree with $\mathcal{M}_0^*$, whereas atomic recall measures the fraction of reference fields recovered by the parser.

Table VI shows a clear dependence on LLM capability. GPT-5.6 Terra achieves 98% exact- $\mathcal{M}_0$ recovery on all three splits, whereas Qwen3.5-9B exhibits lower exact recovery and greater sensitivity to the structural split. The high atomic precision and recall of the compact models nevertheless indicate that most operator-specified fields are recovered correctly even when the complete specification is not an exact match. The larger degradation under the unseen radio-communication register therefore reflects residual semantic sensitivity of the underlying LLM rather than a failure of the structured output interface.

To isolate the contribution of learned semantic parsing, a deterministic, ontology-aware parser is also evaluated. This baseline uses a fixed maneuver lexicon, ordered clause-matching rules, and regular expressions for preferences and numerical constraints, while leaving unmentioned $\mathcal{M}_0$ fields unspecified. The rules were developed on the 100-command development split and frozen before held-out evaluation; exact- $\mathcal{M}_0$ recovery is 92%, 92%, and 57% on the development, held-out instance, and held-out structural splits, respectively. Figure 6 evaluates the complete pipeline using the rule-based parser and three language-model parsers. All downstream planning modules are held fixed to the configuration established in Sec. V-A, so only the intent parser is varied. Every parsed specification is screened by the same deterministic boundary/path verifier used in RQ3; at $N_{\text{rev}} = 0$, a detected inconsistency is rejected without an additional model call. The Qwen3.5-9B development cell reports 73% reference satisfaction and 76% pipeline completion using the same $N_{\text{rev}} = 0$, $N_{\text{plan}} = 1$ artifact as the nominal baseline in Figs. 7 and 8. The figure reports the reference- $\mathcal{M}_0^*$ satisfaction rate, defined as the fraction of generated

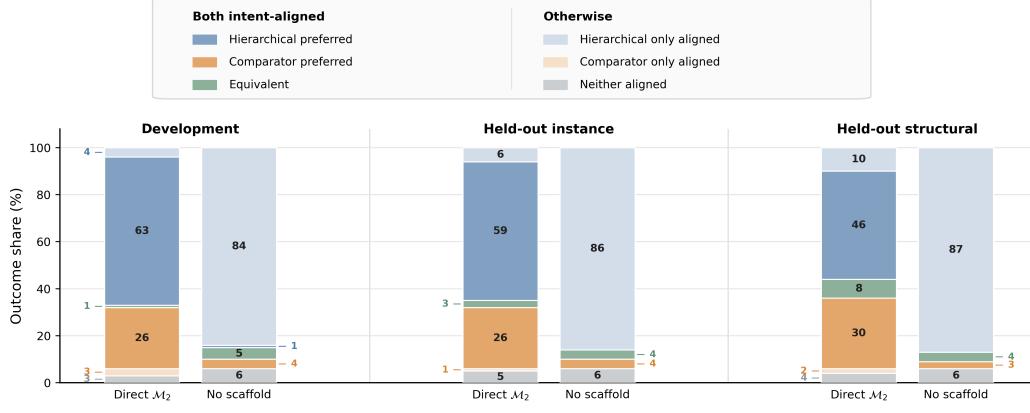


Fig. 5: Architecture comparison against the proposed hierarchical architecture, tested with three dataset splits.

trajectories that satisfy the reference mission specification $\mathcal{M}_0^*$, regardless of whether the intermediate parsed specification exactly matches $\mathcal{M}_0^*$. The pipeline completion rate is also reported as a supplemental measure of downstream execution success. Passing the reference $\mathcal{M}_0^*$ directly to the downstream planner provides an empirical ceiling that accounts for failures originating from the behavior-duration, waypoint-generation, and trajectory optimization stages themselves. With GPT-5.6 Terra, the language-conditioned pipeline closely approaches this reference across all three command sets, showing that a sufficiently capable semantic backend can recover the operator intent with little additional end-to-end degradation. The compact models retain relatively strong performance on commands expressed in the linguistic styles represented during prompt development, but exhibit greater sensitivity to the unseen radio-communication register. The rule-based parser achieves reference- $\mathcal{M}_0^*$ satisfaction and pipeline-completion rates of 87% and 92%, respectively, on both the development and instance-level splits, which share semantic frames and differ only in instance values, exceeding both compact models; however, it declines to 60% and 85% on the structural split, falling below both compact models in reference satisfaction. This degradation illustrates that hand-engineered rules can work well within the controlled grammar used for development but remain fragile to changes in linguistic register and phrasing. The gap between pipeline completion and reference-intent satisfaction is particularly informative in these cases: successful trajectory optimization does not by itself imply that the generated trajectory realizes the commanded mission semantics.

D. RQ3: Test-Time Compute Across the Planning Hierarchy

The results in RQ2 show that the frontier LLM approaches the ceiling of the semantic understanding of the proposed framework, whereas smaller models retain a non-negligible error rate in intent parsing. The final RQ therefore investigates whether additional allocation of

test-time computation can improve performance without retraining or increasing the baseline model size, and how its effect depends on where that computation is allocated within the planning hierarchy. In this subsection, Qwen3.5-9B is used as a representative compact model, and the 100-command development split is used for the natural-language command data. Test-time compute is considered at two levels of the hierarchy: computation at intent parsing through verifier-guided revision and multi-hypothesis generation, and computation at downstream behavior planning through evaluation of additional behavior-duration candidates.

1. Intent Parsing

To provide additional test-time compute at the intent-parsing stage, two methods are considered in this paper. The first is a verifier-guided revision mechanism, in which each proposed partial mission specification $\mathcal{M}_0$ is checked for compatibility with the required `MissionConfig` schema, supported ontology, basic structural constraints, and selected movement patterns that can be deterministically inferred from explicitly stated waypoint and maneuver sequences, as detailed in the Supplementary Material Sec. E. The verifier is active for every revision budget: at $N_{\text{rev}} = 0$, a detected inconsistency causes rejection, whereas for $N_{\text{rev}} > 0$, the previous $\mathcal{M}_0$, together with localized verifier feedback, is returned to the same LLM for correction; otherwise, the parser terminates immediately. Consequently, N_{rev} specifies a maximum revision budget rather than a fixed number of LLM calls, allowing additional computation to be invoked only when the verifier identifies a correctable inconsistency.

Figure 7a shows the resulting trade-off between revision budget and intent-parsing accuracy. Exact- $\mathcal{M}_0$ recovery increases from 75% without revision to 88% at $N_{\text{rev}} = 2$, after which no further improvement is observed through $N_{\text{rev}} = 5$. The mean number of LLM calls per command similarly saturates beyond $N_{\text{rev}} = 2$, indicating that additional revision opportunities are rarely invoked once the deterministic verifier no longer detects action-

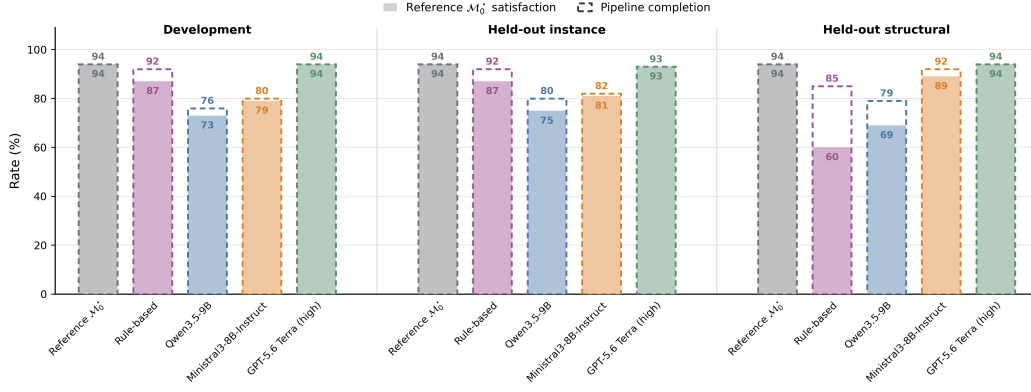


Fig. 6: End-to-end mission satisfaction and pipeline completion across semantic parsers on 100 commands per split. All parsed specifications are screened by the same deterministic boundary/path verifier at $N_{\text{rev}} = 0$.

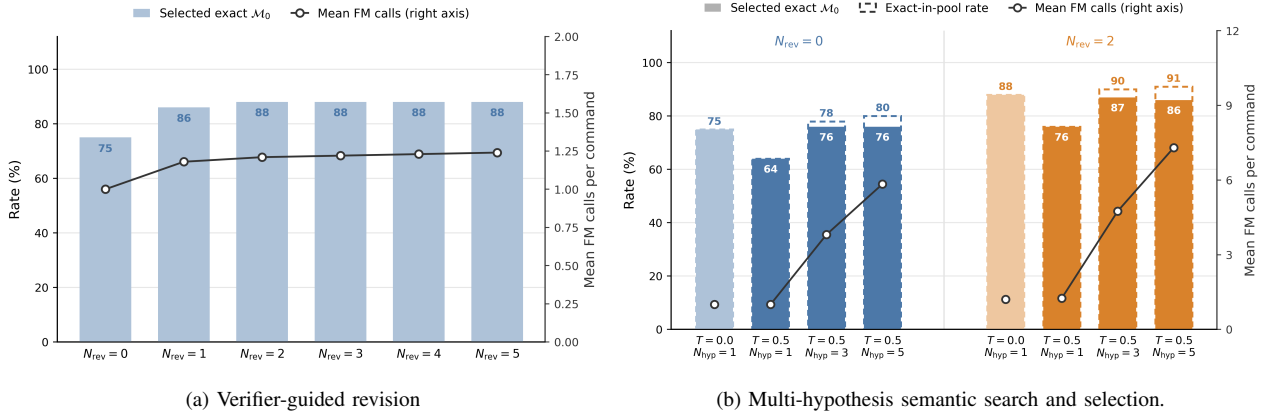


Fig. 7: Effect of semantic test-time compute on intent parsing with Qwen3.5-9B and the development-split commands.

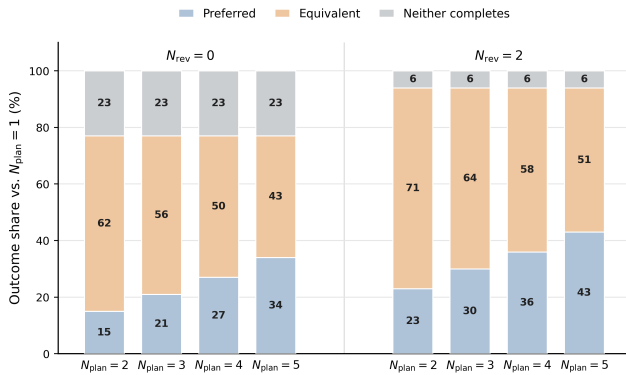


Fig. 8: Effect of planning-search budget over the top- N_{plan} behavior-duration candidates ranked by Q_{ψ} .

able errors. The remaining errors are therefore primarily semantically incorrect but structurally valid interpretations that are not observable by the verifier.

The second method expands the semantic search by generating multiple candidate mission specifications. Specifically, the generation temperature of the LLM, denoted as T , is increased to promote diversity; N_{hyp}

candidate $\mathcal{M}_0$ specifications are generated independently, and a final aggregation query selects a single candidate for downstream planning. In this experiment, greedy decoding ($T = 0$) provides the nominal baseline, while multi-hypothesis generation uses $T = 0.5$ to induce diversity of the output. The same Qwen3.5-9B backend is used for both hypothesis generation and final aggregation.

Figure 7b evaluates this approach for $N_{\text{hyp}} \in \{1, 3, 5\}$, both without revision and with $N_{\text{rev}} = 2$; it also reports the mean LLM calls per command, which includes both hypothesis-generation/revision calls and the final aggregation call. It is observed that a single stochastic hypothesis is less reliable than greedy decoding, reducing exact- $\mathcal{M}_0$ in both $N_{\text{rev}} = 0$ and $N_{\text{rev}} = 2$. Increasing the hypothesis budget to $N_{\text{hyp}} = 3$ recovers this loss, reaching 76% and 87% exact recovery, respectively. However, further increasing the budget to $N_{\text{hyp}} = 5$ provides no additional gain in the selected specification despite the increased inference cost. Importantly, the candidate pool itself continues to improve with additional hypotheses, where the coverage of the exact $\mathcal{M}_0^*$ (i.e., probability that generated hypotheses contain $\mathcal{M}_0^*$) increases as the number of the generated $\mathcal{M}_0$ hypotheses increases. Thus, stochastic multi-hypothesis generation increases the prob-

ability that a correct interpretation is available, but the additional diversity does not necessarily translate into a better final selection. Beyond a modest hypothesis budget, the limiting factor therefore shifts from candidate generation toward aggregation and selection rather than a lack of candidate coverage.

2. Behavior-Duration Planning

A second axis of test-time computation expands the downstream search over behavior–duration plans for a fixed parsed specification $\mathcal{M}_0$. Unlike the semantic mechanisms above, this search does not improve intent alignment; instead, it seeks a more preferable trajectory among admissible realizations of the recovered mission specification. For each $\mathcal{M}_0$, graph-valid behavior–duration candidates are first ranked by the surrogate Q_ψ as described in Sec. IV-D4. The top N_{plan} candidates are then instantiated using the minimum-displacement waypoint heuristic and evaluated through the trajectory optimization under the variable-waypoint contract. Among the successfully optimized candidates, the final plan is selected according to the realized lexicographic preference. Thus, increasing N_{plan} expends additional compute in trajectory optimization rather than additional LLM calls.

Figure 8 evaluates this planning-search budget relative to the nominal $N_{\text{plan}} = 1$ solution. Because the candidate sets are nested and the best successfully optimized trajectory is retained, increasing N_{plan} cannot degrade the realized preference relative to $N_{\text{plan}} = 1$. Increasing the search budget progressively improves trajectory quality: evaluating the top five candidates yields a preferred solution for 34% ($N_{\text{rev}} = 0$) and 43% ($N_{\text{rev}} = 2$) of commands relative to using only the top-ranked candidate. The improvement persists under both revision budgets, indicating that verifier-guided semantic refinement and downstream planning search act on largely distinct aspects of the pipeline. Together with Fig. 3, these results show that Q_ψ provides an effective but coarse filter for promising behavior–duration candidates, while full trajectory optimization of multiple top-ranked candidates can correct residual surrogate-ranking errors and improve the final realized preference. This benefit is therefore complementary to semantic test-time computation, where revision improves the probability of recovering the intended mission specification, whereas behavior-planning search improves trajectory quality after semantic grounding.

VI. Conclusion

This paper introduced a hierarchical TAMP architecture that grounds LLM reasoning in spacecraft-specific behavior abstractions, operational constraints, and orbital dynamics to enable language-commanded trajectory generation for rendezvous and proximity operations. Through a structured mission representation, a graph of reusable spacecraft behaviors, and domain-specific planning modules, the architecture progressively translates high-level operator intent into a complete, verifiable mission speci-

fication and, ultimately, a trajectory satisfying the adopted dynamics and safety constraints. This hierarchy separates semantic grounding, completion of unspecified mission decisions, and physical trajectory verification while preserving operator-provided information throughout the planning process. Numerical experiments demonstrated the advantages of the hierarchical architecture over one-shot LLM-based planning, characterized how structured intent recognition and end-to-end mission satisfaction vary with model capability, and showed how test-time computation at different layers can improve semantic reliability and realized trajectory quality. Together, these results establish the proposed framework as a step toward scalable, auditable human–spacecraft interfaces for future agentic spacecraft operations. Future work will extend the framework to multi-spacecraft coordination and closed-loop execution with online replanning.

Acknowledgment

AI tools were used for brainstorming, coding assistance, and language polishing.

REFERENCES

- [1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “React: Synergizing reasoning and acting in language models,” *arXiv preprint arXiv:2210.03629*, 2022.
- [2] D. A. Boiko, R. MacKnight, B. Kline, and G. Gomes, “Autonomous chemical research with large language models,” *Nature*, vol. 624, no. 7992, pp. 570–578, 2023.
- [3] A. Brohan, Y. Chebotar, C. Finn, K. Hausman, A. Herzog, D. Ho, J. Ibarz, A. Irpan, E. Jang, R. Julian, *et al.*, “Do as I can, not as I say: Grounding language in robotic affordances,” in *Conference on robot learning*, pp. 287–318, PMLR, 2023.
- [4] Y. Wang, W. Luo, J. Bai, Y. Cao, T. Che, K. Chen, Y. Chen, J. Diamond, Y. Ding, W. Ding, *et al.*, “Alpamayo-r1: Bridging reasoning and action prediction for generalizable autonomous driving in the long tail,” *arXiv preprint arXiv:2511.00088*, 2025.
- [5] J. Telaar, I. Ahrns, S. Estable, W. Rackl, M. De Stefano, R. Lampariello, N. Santos, P. Serra, M. Canetti, F. Ankersen, and J. Gil-Fernández, “GNC architecture for the e.Deorbit mission,” in *7th European Conference for Aeronautics and Space Sciences*, 2017.
- [6] Y. Takubo and S. D’Amico, “Intent-aligned autonomous spacecraft guidance via reasoning models,” in *IEEE Conf. on Computer Vision and Pattern Recognition, Workshop on Artificial Intelligence for Space*, 2026.
- [7] Y. Zhang, B. Jiao, and Z. Dang, “A large language model-based approach to spacecraft formation mission planning,” *IFAC-PapersOnLine*, vol. 59, no. 20, pp. 2166–2170, 2025. 23th IFAC Symposium on Automatic Control in Aerospace ACA 20255.
- [8] M. Foutter, D. Gammelli, J. Kruger, E. Foss, P. Bhoj, T. Guffanti, S. D’Amico, and M. Pavone, “Space-llava: A vision-language model adapted to extraterrestrial applications,” in *2025 IEEE Aerospace Conference*, pp. 1–23, 2025.
- [9] N. Muscettola, P. P. Nayak, B. Pell, and B. C. Williams, “Remote agent: To boldly go where no AI system has gone before,” *Artificial Intelligence*, vol. 103, no. 1–2, pp. 5–47, 1998.
- [10] S. Chien, R. Sherwood, D. Tran, B. Cichy, G. Rabideau, R. Castano, A. Davies, D. Mandl, S. Frye, B. Trout, S. Shulman, and D. Boyer, “Using autonomy flight software to improve science return on Earth Observing One,” *Journal of Aerospace*

- Computing, Information, and Communication*, vol. 2, no. 4, pp. 196–216, 2005.
- [11] T. Guffanti, T. Bell, S. Y. Low, M. Murray-Cooper, and S. D’Amico, “Autonomous guidance navigation and control of the visors formation-flying mission,” in *2023 AAS/AIAA Astrodynamics Specialist Conference, Big Sky, MT*, 2023.
 - [12] J. J. Kruger, T. Guffanti, T. H. Park, M. Murray-Cooper, S. Y. Low, T. Bell, S. D’Amico, C. W. Roscoe, and J. Westphal, “Adaptive end-to-end architecture for autonomous spacecraft navigation and control during rendezvous and proximity operations,” in *AIAA SCITECH 2024 Forum*, p. 0430, 2024.
 - [13] D. F. Lawden, *Optimal trajectories for space navigation*. 1963.
 - [14] D. Malyuta, T. P. Reynolds, M. Szmuk, T. Lew, R. Bonalli, M. Pavone, and B. Açıkmeşe, “Convex Optimization for Trajectory Generation: A Tutorial on Generating Dynamically Feasible Trajectories Reliably and Efficiently,” *IEEE Control Systems Magazine*, vol. 42, no. 5, pp. 40–113, 2022.
 - [15] S. Banerjee, T. Lew, R. Bonalli, A. Alfaadhel, I. A. Alomar, H. M. Shageer, and M. Pavone, “Learning-based Warm-Starting for Fast Sequential Convex Programming and Trajectory Optimization,” in *IEEE Aerospace Conference*, pp. 1–8, 2020.
 - [16] T. Kim, P. Elango, D. Malyuta, and B. Açıkmeşe, “Guided policy search using sequential convex programming for initialization of trajectory optimization algorithms,” in *2022 American Control Conference (ACC)*, pp. 3572–3578, IEEE, 2022.
 - [17] T. Guffanti, D. Gammelli, S. D’Amico, and M. Pavone, “Transformers for trajectory optimization with application to spacecraft rendezvous,” in *IEEE Aerospace Conference*, pp. 1–13, 2024.
 - [18] Y. Takubo, T. Guffanti, D. Gammelli, M. Pavone, and S. D’Amico, “Towards robust spacecraft trajectory optimization via transformers,” in *IEEE Aerospace Conference*, 2025.
 - [19] A. Carrasco, V. Rodriguez-Fernandez, and R. Linares, “Large language models as autonomous spacecraft operators in kerbal space program,” *Advances in Space Research*, 2025.
 - [20] A. Jain and R. Linares, “Autonomous reasoning for spacecraft control: A large language model framework with group relative policy optimization,” *arXiv preprint arXiv:2601.04334*, 2026.
 - [21] Y. Chen, R. Gandhi, Y. Zhang, and C. Fan, “NI2tl: Transforming natural languages to temporal logics using large language models,” *arXiv preprint arXiv:2305.07766*, 2023.
 - [22] E. Brosius, Y. Takubo, D. Gammelli, S. D’Amico, and M. Pavone, “Semantic constraint synthesis for adaptive trajectory optimization via large language models,” *arXiv preprint arXiv:2606.04123*, 2026.
 - [23] Y. Takubo, A. Dwivedi, S. Ramkumar, L. A. Pabon, D. Gammelli, M. Pavone, and S. D’Amico, “Language-conditioned safe trajectory generation for spacecraft rendezvous,” *Journal of Guidance, Control, and Dynamics*, pp. 1–20, 2026.
 - [24] C. R. Garrett, R. Chitnis, R. Holladay, B. Kim, T. Silver, L. P. Kaelbling, and T. Lozano-Pérez, “Integrated task and motion planning,” *Annual review of control, robotics, and autonomous systems*, vol. 4, no. 1, pp. 265–293, 2021.
 - [25] C. R. Garrett, T. Lozano-Pérez, and L. P. Kaelbling, “PDDLStream: Integrating symbolic planners and blackbox samplers via optimistic adaptive planning,” in *Proceedings of the International Conference on Automated Planning and Scheduling*, vol. 30, pp. 440–448, 2020.
 - [26] M. Toussaint, “Logic-geometric programming: An optimization-based approach to combined task and motion planning,” in *Proceedings of the Twenty-Fourth International Joint Conference on Artificial Intelligence*, pp. 1930–1936, 2015.
 - [27] B. Kim, L. Shimanuki, L. P. Kaelbling, and T. Lozano-Pérez, “Representation, learning, and planning algorithms for geometric task and motion planning,” *The International Journal of Robotics Research*, vol. 41, no. 2, pp. 210–231, 2022.
 - [28] L. Xu, T. Ren, G. Chalvatzaki, and J. Peters, “Accelerating integrated task and motion planning with neural feasibility checking,” *arXiv preprint arXiv:2203.10568*, 2022.
 - [29] T. Silver, R. Chitnis, J. Tenenbaum, L. P. Kaelbling, and T. Lozano-Pérez, “Learning symbolic operators for task and motion planning,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2021.
 - [30] W. Huang, P. Abbeel, D. Pathak, and I. Mordatch, “Language models as zero-shot planners: Extracting actionable knowledge for embodied agents,” in *International conference on machine learning*, pp. 9118–9147, PMLR, 2022.
 - [31] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, “Code as policies: Language model programs for embodied control,” in *IEEE International Conference on Robotics and Automation*, 2023.
 - [32] K. Lin, C. Agia, T. Migimatsu, M. Pavone, and J. Bohg, “Text2motion: From natural language instructions to feasible plans,” *Autonomous Robots*, vol. 47, no. 8, pp. 1345–1365, 2023.
 - [33] Y. Chen, J. Arkin, C. Dawson, Y. Zhang, N. Roy, and C. Fan, “AutoTAMP: Autoregressive task and motion planning with LLMs as translators and checkers,” in *IEEE International Conference on Robotics and Automation*, pp. 6695–6702, 2024.
 - [34] S. Kambhampati, K. Valmeekam, L. Guan, M. Verma, K. Stechly, S. Bhambri, L. P. Saldyt, and A. B. Murthy, “Position: LLMs can’t plan, but can help planning in LLM-Modulo frameworks,” in *Proceedings of the 41st International Conference on Machine Learning*, pp. 22895–22907, PMLR, 2024.
 - [35] J. A. Starek, E. Schmerling, G. D. Maher, B. W. Barbee, and M. Pavone, “Fast, safe, propellant-efficient spacecraft motion planning under Clohessy–Wiltshire–Hill dynamics,” *Journal of Guidance, Control, and Dynamics*, vol. 40, no. 2, pp. 418–438, 2017.
 - [36] T. Deka and J. McMahon, “Astrodynamics-informed kinodynamic sampling-based motion planning for relative spacecraft motion,” *Journal of Guidance, Control, and Dynamics*, vol. 46, no. 12, pp. 2330–2345, 2023.
 - [37] S. D’Amico, *Autonomous Formation Flying in Low Earth Orbit*. PhD Thesis, Delft University, 2010.
 - [38] A. W. Koenig, T. Guffanti, and S. D’Amico, “New state transition matrices for spacecraft relative motion in perturbed orbits,” *Journal of Guidance, Control, and Dynamics*, vol. 40, no. 7, pp. 1749–1768, 2017.
 - [39] T. Guffanti and S. D’Amico, “Passively Safe and Robust Multi-Agent Optimal Control with Application to Distributed Space Systems,” *AIAA Journal of Guidance, Control, and Dynamics*, vol. 46, no. 8, pp. 1448–1469, 2023.
 - [40] P. Elango, D. Luo, A. G. Kamath, S. Uzun, T. Kim, and B. Açıkmeşe, “Continuous-time successive convexification for constrained trajectory optimization,” *Automatica*, vol. 180, p. 112464, 2025.
 - [41] K. Oguri, “Successive convexification with feasibility guarantee via augmented lagrangian for non-convex optimal control problems,” in *Proc. IEEE Conf. on Decision and Control*, pp. 3296–3302, IEEE, 2023.
 - [42] J. Kwok, S. Li, P. Atreya, Y. Liu, Y. Jiang, C. Finn, M. Pavone, I. Stoica, and A. Mirhoseini, “Llm-as-a-verifier: A general-purpose verification framework,” *arXiv preprint arXiv:2607.05391*, 2026.
 - [43] OpenAI, “GPT-5.6 System Card.” OpenAI Deployment Safety Hub, July 2026. Accessed: 2026-08-13.
 - [44] Qwen Team, “Qwen3.5: Towards native multimodal agents,” February 2026.
 - [45] A. H. Liu, K. Khandelwal, S. Subramanian, V. Jouault, A. Rastogi, A. Sadé, A. Jeffares, A. Jiang, A. Cahill, A. Gavaudan, et al., “Ministral 3,” *arXiv preprint arXiv:2601.08584*, 2026.



Yuji Takubo (Student Member, IEEE) is a Ph.D. candidate in the Space Rendezvous Lab at Stanford University. He received the M.S. in Aeronautics and Astronautics from Stanford University (2025) and the B.S. in Aerospace Engineering from the Georgia Institute of Technology (2023). He is a recipient of the NASA Jet Propulsion Laboratory Graduate Fellowship (2023) and the Ezoe Memorial Recruit Foundation Fellowship (2020-). His research

focuses on optimization and astrodynamics, particularly with emphasis on distributed space systems.



Daniele Gammelli is the Research Director of the Machine Intelligence for Robot Autonomy Laboratory at the Italian Institute of Artificial Intelligence (IIAI), and a Researcher in the Department of Aeronautics and Astronautics at Stanford University. Since 2022, he has been a Research Fellow at the Center for Aerospace Autonomy Research (CAESAR) at Stanford and, until 2025, at the Center for Automotive Research at Stanford (CARS). He received his

Ph.D. in Machine Learning and Mathematical Optimization from the Technical University of Denmark (DTU) in 2022, where his doctoral thesis was nominated for the DTU Best Thesis of the Year Award and the DTU Young Researcher Award. Dr. Gammelli's research focuses on developing the algorithmic foundations and system-level methodologies that enable AI-powered autonomous systems to operate safely, efficiently, and reliably in high-stakes environments, with an emphasis on aerospace autonomy and next-generation mobility systems.



Marco Pavone (Member, IEEE) is a Professor of Aeronautics and Astronautics at Stanford University, where he directs the Autonomous Systems Laboratory and the Center for Automotive Research at Stanford. He is also a Distinguished Research Scientist at NVIDIA where he leads autonomous vehicle research. Before joining Stanford, he was a Research Technologist within the Robotics Section at the NASA Jet Propulsion Laboratory. He received

a Ph.D. degree in Aeronautics and Astronautics from the Massachusetts Institute of Technology in 2010. His main research interests are in the development of methodologies for the analysis, design, and control of autonomous systems, with an emphasis on self-driving cars, autonomous aerospace vehicles, and future mobility systems. He is a recipient of a number of awards, including a Presidential Early Career Award for Scientists and Engineers from President Barack Obama, an Office of Naval Research Young Investigator Award, a National Science Foundation Early Career (CAREER) Award, a NASA Early Career Faculty Award, and an Early-Career Spotlight Award from the Robotics Science and Systems Foundation. He was identified by the American Society for Engineering Education (ASEE) as one of America's 20 most highly promising investigators under the age of 40. His work has been recognized with best paper nominations or awards at a number of venues, including the European Conference on Computer Vision, the IEEE International Conference on Robotics and Automation, the European Control Conference, the IEEE International Conference on Intelligent Transportation Systems, the Field and Service Robotics Conference, the Robotics: Science and Systems Conference, and the INFORMS Annual Meeting.



Simone D'Amico (Member, IEEE) Associate Professor of Aeronautics and Astronautics (AA) and Professor of Geophysics (by Courtesy). He is the Founding Director of the Stanford Space Rendezvous Laboratory, Director of the Undergraduate Program in AA, and Founding Co-Director of the Center for AeroSpace Autonomy Research (CAESAR) at Stanford. He is a W.M. Keck Faculty Scholar in the School of Engineering and Hoover Science

Fellow. He has 20+ years of experience in research and development of autonomous spacecraft and distributed space systems, including multi-agent architectures such as "rendezvous, proximity operations, and capture", "satellite formation-flying and swarms", "fractionated spacecraft", and "mega-constellations". He developed the GNC system of several such missions (e.g., TanDEM-X, PRISMA) and is currently the institutional PI of four autonomous satellite swarms funded by NASA and NSF with one of them (NASA Starling) operational in orbit. Besides academia, Dr. D'Amico is the Chief Science Officer (CSO) of EraDrive. He is in the Advisory Board of four space start-ups focusing on distributed space systems for future applications in SAR remote sensing, orbital lifetime prolongation, and space-based solar power. He received the B.S. and M.S. degrees from Politecnico di Milano (2003) and the Ph.D. degree from Delft University of Technology (2010). Before Stanford, Dr. D'Amico was research scientist and team leader at the German Aerospace Center (DLR). He was the recipient of several awards, including the 2020 IEEE M. Barry Carlton Award, Best Paper Awards at AMOS (2025), IAF (2022), IEEE (2021), AIAA (2021), AAS (2019) conferences, the Leonardo 500 Award by the Leonardo da Vinci Society/ISSNAF (2019), FAI/NAA's Group Diploma of Honor (2018), DLR's Sabbatical/Forschungssemester (2012) and Wissenschaft Preis (2006), and NASA's Group Achievement Awards for the STARLING (2024) and GRACE mission (2004).

Supplementary Material

OrbitTAMP: Grounding Language Models for Task and Motion Planning in Spacecraft Rendezvous

Yuji Takubo¹, Daniele Gammelli^{1,2}, Marco Pavone^{1,3}, and Simone D’Amico¹

A. Learning-based Waypoint Generation

This subsection discusses the detailed implementation of the learning-based waypoint generation module.

First, training data are generated by solving the trajectory optimization problem (cf. Eq. 11) for randomized graph-valid behavior-duration sequences, scenarios, and waypoint sampling within each waypoint domain. In particular, the waypoint for these rows is drawn from an 11-point grid spanning each domain; the initial waypoint w_0 for these rows is drawn from the same on-grid domain sampler. A SCP solve optimizes $(\delta\lambda, \delta e_y, \delta i_y)$ jointly within the selected waypoint domains. In this case study, 20,000 data rows are generated, and the optimized waypoint sequences from converged rows form the imitation targets. The rows are split randomly, with 80%/10%/10% used for training, validation, and testing.

The model architecture adopts a multilayer perceptron (MLP) with three hidden layers of 256 units each. In order to account for the disconnected waypoint domains, each disconnected coordinate is represented by a magnitude logit and a binary sign. Namely, at each phase $m = 1, \dots, M$, it outputs Gaussian means and standard deviations for the three continuous variables and logits for the two signs:

$$z_{\lambda,k} = \text{logit}\left(\frac{a\delta\lambda_k - \ell_{\lambda,k}}{u_{\lambda,k} - \ell_{\lambda,k}}\right), \quad z_{v,k} = \text{logit}\left(\frac{|a\delta v_k| - \ell_{v,k}}{u_{v,k} - \ell_{v,k}}\right), \quad (\text{A1a})$$

$$s_{v,k} = \mathbb{I}[a\delta v_k \geq 0], \quad v \in \{e_y, i_y\}. \quad (\text{A1b})$$

Here, $[\ell_{\lambda,k}, u_{\lambda,k}]$ is the connected along-track interval, and $[\ell_{v,k}, u_{v,k}]$ is the admissible magnitude interval. The inverse map uses an affine sigmoid, which guarantees that the prediction stays inside the graph-prescribed domain.

The model is trained using unweighted imitation learning, supervised on the converged rows. Each training example already contains one sampled graph-valid behavior-duration sequence $(b_{1:M}, d_{1:M})$ and its optimized waypoint targets $w_{1:M}^*$. The loss uses Gaussian negative log-likelihood (NLL) for the continuous targets and binary cross-entropy (BCE) for the sign targets. The per-example loss and unweighted

minibatch objective are

$$\begin{aligned} \mathcal{L}_\phi &= \frac{1}{|\mathcal{I}|} \sum_{i \in \mathcal{I}} \ell_\phi^{(i)}. \\ \ell_\phi &= \frac{1}{3M} \sum_{k=1}^M \sum_{c=1}^3 \text{NLL}_{\mathcal{N}}(\tilde{z}_{kc}; \mu_{\phi,kc}, \sigma_{\phi,kc}) \\ &\quad + \frac{1}{2M} \sum_{k=1}^M \sum_{c=1}^2 \text{BCE}(s_{kc}; \pi_{\phi,kc}), \end{aligned} \quad (\text{A2})$$

where $\mathcal{I}$ is the minibatch index set, and each component $\tilde{z}_{kc} = (z_{kc} - \bar{z}_{kc})/\tau_{kc}$ is normalized using the training-set mean $\bar{z}_{kc}$ and standard deviation τ_{kc} . The quantities $\mu_{\phi,kc}$ and $\sigma_{\phi,kc}$ are the corresponding predicted Gaussian parameters, and $\pi_{\phi,kc}$ is the softmax probability of the positive-sign class. The factors $3M$ and $2M$ average the continuous and sign losses over the M selected phases, so two- and three-phase data receive equal weight.

B. Learning-based Behavior-Duration Planning

This subsection discusses the detailed implementation of the learning-based behavior-duration planning module.

First, the dataset generation proceeds similarly to learning-based waypoint generation, where data are generated by solving Eq. 11 for randomized graph-valid behavior-duration sequences and scenarios. However, the waypoint corresponding to the sampled behavior-duration pair is selected using the corresponding waypoint policy π_w . Namely, the trained model Q_ψ is strictly dependent on how the waypoint is placed.

The metric surrogate model is formulated as an MLP and a gated recurrent unit (GRU) with a 128-dimensional hidden state. The final output comprises a scalar convergence prediction $\hat{p}_{\text{conv}} \in [0, 1]$ and a metric prediction $\hat{q} \in \mathbb{R}^{N_P}$. To predict the convergence of the subsequent optimization, both converged and nonconverged rows are used for training.

The loss combines a BCE term on predicted convergence with a Huber loss over the realized metrics; the latter is evaluated only on converged rows:

$$\begin{aligned} \mathcal{L}_\psi &= \frac{1}{|\mathcal{I}|} \sum_{i \in \mathcal{I}} \text{BCE}(\hat{p}_{\text{conv}}^{(i)}, y_{\text{conv}}^{(i)}) \\ &\quad + \frac{\lambda_q}{N_P |\mathcal{I}_{\text{conv}}|} \sum_{i \in \mathcal{I}_{\text{conv}}} \sum_{j=1}^{N_P} \text{Huber}_1(\hat{q}_j^{(i)} - \hat{q}_j^{(i)}), \end{aligned} \quad (\text{A3})$$

where $\mathcal{I}_{\text{conv}} = \{i \in \mathcal{I} : y_{\text{conv}}^{(i)} = 1\}$ denotes the converged rows, where $y_{\text{conv}} \in \{0, 1\}$ is the binary indicator of

¹ Department of Aeronautics and Astronautics, Stanford University, Stanford, CA, 94305. {ytakubo, gammelli, pavone, damicos}@stanford.edu.

² Italian Institute of Artificial Intelligence (IAI), Turin, Italy

³ NVIDIA Research, Santa Clara, CA 95051

numerical convergence in the optimization, and λ_q is a weighting parameter. The value $\tilde{q}_j$ is normalized using the median and interquartile range (IQR) of metric q_j computed from the converged training rows, and $\hat{q}_j$ is its prediction. For the safety margin, the raw value is clipped at 25 m before scaling to improve prediction near the boundary. Huber₁ is the Huber loss with unit transition width, which prevents a few large residuals from dominating the gradient. Thus, the convergence head uses every row, whereas the metric loss uses only converged rows. Optimization convergence probabilities are Platt-calibrated on the validation set. In this case study, 20,000 data rows are generated, and the dataset is partitioned randomly into an 80%/10%/10% split.

At deployment, all graph-valid behavior-duration candidates consistent with $\mathcal{M}_0$ are enumerated (cf. Table II) and evaluated by Q_ψ in one batched forward pass. Candidates with predicted convergence probability below $p_{\min} = 0.5$ are removed, and the remainder are ranked by ρ_P to determine the top-ranked behavior-duration plan.

C. Prompt Structure

The following box summarizes the prompt supplied to the LLM rather than reproducing it verbatim. The behavior and waypoint ontology included in the prompt is the same as that defined in Tables I and II. In the actual implementation, this template is augmented with domain-specific interpretation rules, summarized by the parsing principles below, and scenario-specific grounding based on the initial waypoint domain.

Condensed initial parsing prompt

```
System:
Parse the command into a partial MissionConfig.
Preserve explicit operator intent, leave
unsupported fields unspecified, and return one
JSON object without prose.

User:
Output schema:
{
  "preferences": {"priority": [...]},
  "hard_constraints": {...},
  "b_seq": [...], "d_seq": [...],
  "tof_orbit": null,
  "tof_bound": {"lower_orbit": null,
               "upper_orbit": null},
  "x_seq": [...]
}

Parsing principles:
1. Extract only stated intent; do not complete the
   mission.
2. Preserve the number and order of stated phases.
3. For a multi-phase command, represent each stated
   phase with corresponding behavior, duration, and
   waypoint entries, using unspecified values when
   information is omitted.
4. Distinguish where a maneuver starts from where the
   operator requests it to end.
5. Do not add intermediate phases or behaviors that
   the operator did not state.
6. Use only the supplied ontology, metrics, and
   constraints.

<behavior and waypoint ontology>
start_domain = <domain>
operator_command = <command>
```

TABLE I: Illustrative renderings of the same reference mission intent across the supported language styles.

Style	Command realization
Formal	Circumnavigate the target through the near-target operating region. Prioritize maximizing observation quality over minimizing completion time.
Natural	Move the chaser around the target along a circumnavigation path. Give observation quality higher priority than completion time.
Concise	Chaser task: target circumnavigation. Priority order: observation quality, then completion time.
Radio	Circumnavigate target, near-field. Prioritize observation quality over completion time, all phases.

TABLE II: Command-benchmark splits used for language-conditioned evaluation.

Split	Mission instances	Language styles
Development	Development sample	Formal, natural, concise
Held-out instance	Independently sampled	Formal, natural, concise (seen)
Held-out structural	Independently sampled	Radio register (unseen)

If verification fails, the following message is appended to the original conversation, so the model retains the schema, ontology, command, and previous response.

Verifier-guided revision message

```
Previous response:
<candidate MissionConfig>

Verifier feedback:
<list of deterministic errors>

Return a complete corrected MissionConfig as JSON.
Preserve fields not implicated by the errors and
do not add unstated mission intent.
```

D. Command-Benchmark Construction

Four linguistic styles are used to render the operator commands: formal, natural, concise, and radio. Table I illustrates these styles using the same underlying reference mission intent $\mathcal{M}_0^*$. The linguistic realization changes across styles while preserving the supported mission semantics and behavior-waypoint ontology.

Table II summarizes their use across the three benchmark splits. The development and held-out-instance splits use the formal, natural, and concise styles, with the latter containing independently-sampled mission specifications. The held-out-structural split also uses independently sampled mission specifications, but renders them exclusively in the unseen radio register to introduce a linguistic-structure shift.

E. Verification of $\mathcal{M}_0$

Each $\mathcal{M}_0$ proposed during intent parsing is checked before acceptance. It must contain the required MissionConfig fields with the correct data types, use only the behaviors, waypoint regions, metrics, and constraints defined in

this paper, and contain valid and consistent time bounds. Relative-state waypoints must contain six coordinates, and the behavior, duration, and waypoint sequences must contain the same number of phase entries.

At every revision budget, a small deterministic check also compares the proposed $\mathcal{M}_0$ with certain movement sequences explicitly stated in the command. It applies only to (i) an ordered list of waypoint regions, (ii) an approach followed by departure, or (iii) a direct cross-side flyby or fast passage. Expressions such as F_+ , E_0 , F_- , and their V -bar equivalents are matched to the waypoint regions in Table I, and the behavior–waypoint graph identifies the behavior associated with each stated movement. For example, $F_+ \rightarrow E_0 \rightarrow F_-$ means that the first waypoint is in E_0 and the second is in F_- ; F_+ is the initial region and is therefore not repeated as a waypoint. At $N_{\text{rev}} = 0$, any disagreement causes rejection; at $N_{\text{rev}} > 0$, it is returned to the LLM for revision. Commands outside these three cases receive no movement-specific feedback, and unrelated fields may remain at $\perp$.

After intent parsing, a final check determines whether every populated behavior can connect the preceding waypoint region to the requested next region in the behavior–waypoint graph, beginning from the known initial region. If a behavior is inconsistent but the requested waypoint remains reachable through another allowed behavior, that behavior entry is reset to $\perp$, and the waypoint is preserved. This check does not fill unspecified fields or complete the behavior sequence.

F. SCP Parameters

Table III presents the hyperparameters of $\text{SCvx}\star$ based on the notation in [1]. All optimization variables are normalized based on the warm-start, so that all variables are scaled to lie within $[-1, 1]$.

TABLE III: Hyperparameters of $\text{SCvx}\star$.

Hyperparameter	Value
ϵ	10^{-3}
$\{\rho_0, \rho_1, \rho_2\}$	$\{0.0, 0.25, 0.7\}$
$\{\alpha_1, \alpha_2, \beta, \gamma\}$	$\{2, 2, 1.5, 0.9\}$
$\{r^{(1)}, r_{\min}, r_{\max}\}$	$\{0.2, 10^{-9}, 10\}$
$\{w^{(1)}, w_{\max}\}$	$\{100, 10^{20}\}$
# max. iter	100

REFERENCES

- [1] K. Oguri, “Successive convexification with feasibility guarantee via augmented lagrangian for non-convex optimal control problems,” in *Proc. IEEE Conf. on Decision and Control*, pp. 3296–3302, IEEE, 2023.